

重庆邮电大学智慧访客预约与出入校管理系统 数据库原理课程设计报告

数据库原理期末大作业

2026 年 6 月 23 日

目录

一、 系统定义	3
1.1 本章概述	3
1.2 设计意图	3
1.3 目标功能	3
1.4 数据处理目标	4
1.5 权限访问要求	4
1.6 系统价值	4
二、 需求分析	4
2.1 本章概述	4
2.2 综合需求	4
2.3 功能需求	5
2.4 性能需求	6
2.5 运行需求	6
2.6 安全需求	6
2.7 数据需求	7
2.8 数据流程图	7
2.8.1 顶层数据流程图	7
2.8.2 二层数据流程图	8
2.9 数据字典	8
三、 系统设计	9
3.1 本章概述	9
3.2 概念结构设计说明	9
3.3 E-R 图	9
3.3.1 总体简化 E-R 图	10
3.3.2 核心业务 E-R 图	10
3.3.3 用户权限 E-R 图	11
3.3.4 系统支撑 E-R 图	11
3.4 逻辑结构设计说明	12

3.5	关系模式说明	15
3.6	系统功能模块图	16
3.6.1	系统功能模块图	16
3.7	其它设计图	16
3.7.1	访客预约审批流程图	16
3.7.2	门岗核验入校流程图	17
3.7.3	系统架构图	17
3.7.4	数据库表关系图	18
3.7.5	自动截图与报告生成流程图	19
四、	详细设计	19
4.1	本章概述	19
4.2	SQL-01 查询某访客的历史预约记录	19
4.3	SQL-02 查询某被访人的待确认预约	20
4.4	SQL-03 查询某部门的待审批预约	20
4.5	SQL-04 查询当天已入校访客	21
4.6	SQL-05 查询当天已入校但未离校访客	21
4.7	SQL-06 查询超时未离校访客	22
4.8	SQL-07 按部门统计本月访客数量	22
4.9	SQL-08 按校门统计本月通行数量	23
4.10	SQL-09 查询黑名单访客	23
4.11	SQL-10 查询某预约的完整审批记录	24
4.12	SQL-11 查询某访客的车辆和随行人员	24
4.13	SQL-13 查询今日访客概览	24
4.14	SQL-14 查询近七天访客趋势	25
4.15	SQL-15 查询审批通过率	26
4.16	SQL-18 访问事由分布	26
4.17	SQL-19 审批状态分布	27
4.18	SQL-20 入校离校状态分布	27
4.19	SQL-21 黑名单风险数量	28
4.20	SQL-22 最近操作日志	28
4.21	建表 SQL 摘要	28

五、 系统实现与测试	32
5.1 本章概述	32
5.2 开发平台和工具选择	32
5.3 后端实现说明	33
5.4 前端实现说明	33
5.5 数据库实现说明	33
5.6 自动截图说明	33
5.7 测试方法	34
5.8 代表性运行界面	35
六、 总结	46

一、 系统定义

1.1 本章概述

系统定义用于回答“系统要做什么”。本系统面向重庆邮电大学校园访客管理场景，以数据库为核心组织访客、预约、审批、通行凭证、出入校记录、黑名单、权限和日志数据，形成从访客申请到访问归档的完整信息化闭环。

表 1: 系统基本信息

项目	内容
中文系统名称	重庆邮电大学智慧访客预约与出入校管理系统
英文系统名称	Smart Visitor Reservation and Campus Access Management System of Chongqing University of Posts and Telecommunications
课程设计定位	数据库原理期末大作业，重点展示需求分析、数据流程、E-R 建模、关系模式、SQL 查询和系统实现。
技术形态	Vue 3 前端、Spring Boot 3 后端、MySQL 8 数据库、Playwright 自动截图、LaTeX 自动报告。

1.2 设计意图

系统设计意图是将传统线下访客登记、人工审批和纸质留痕转化为可查询、可追踪、可统计的数据库应用流程。系统以预约申请为主线，将访客身份、被访人确认、部门审批、通行码生成、门岗核验、入校登记、离校登记和访问记录归档统一纳入数据库管理，减少人工信息传递造成的遗漏和延迟。

1.3 目标功能

系统需要支持访客预约、黑名单检查、被访人确认、部门审批、通行凭证生成、门岗核验、入校离校登记、当前在校查询、超时未离校识别、访客记录查询、统计报表、用户角色权限管理和操作日志审计等功能。上述功能均需要与数据库表结构、后端接口和前端页面保持一致。

表 2: 系统服务对象

服务对象	主要使用目标
访客	提交预约申请，查看预约状态和通行凭证。
被访人	确认或拒绝与本人相关的访客预约。
部门审批人员	审批本部门访问申请并填写审批意见。

门岗安保人员	核验通行码，办理入校登记、离校登记和超时处理。
系统管理员	维护用户、角色、权限、部门、校门、黑名单、字典和日志。
校级管理人员	查看全校访客记录和统计报表，为管理决策提供依据。

1.4 数据处理目标

系统的数据处理目标包括：保证访客身份与预约申请可追溯，保证审批记录按流程留痕，保证通行凭证与出入校记录一一关联，保证黑名单风险可在预约和核验阶段被识别，保证统计报表可按日期、部门、校门、状态和访问事由进行汇总分析。

1.5 权限访问要求

系统采用登录认证和 RBAC 权限控制。系统管理员可访问全部功能；被访人仅能处理与本人相关的预约；部门审批人员仅能审批本部门申请；门岗安保人员仅能访问核验、出入校登记和在校访客查询功能；校级管理人员主要查看全校记录和统计报表；访客仅能维护本人预约和查看本人凭证。

1.6 系统价值

从校园管理角度看，系统提升了访客预约审批效率和出入校安全控制能力。从数据库课程设计角度看，系统覆盖实体识别、联系分析、关系模式转换、主外键约束、索引设计、典型 SQL 查询、系统实现和测试截图，能够较完整地体现数据库应用系统设计过程。

二、需求分析

2.1 本章概述

需求分析用于回答“系统必须做什么”。本系统围绕访客预约与出入校管理的完整流程，明确用户角色、功能边界、数据需求、安全要求和运行要求，并通过数据流程图和数据字典为后续数据库建模提供依据。

2.2 综合需求

重庆邮电大学日常教学、科研、合作交流、后勤维修、招聘面试、校友返校等场景中存在大量校外人员入校需求。系统必须支持如下主流程：访客提交预约申请 → 系统检查黑名单 → 被访人确认 → 部门审批 → 生成通行凭证 → 门岗核验 → 入校登记 → 离校

登记 → 访问记录归档 → 查询统计。系统还必须处理黑名单拦截、被访人拒绝、部门审批拒绝、通行码过期、重复离校和超时未离校等异常场景。

表 3: 用户角色与数据访问范围

角色	主要职责	数据访问范围
访客	预约申请、状态查询、通行凭证查看	本人访客信息、本人预约、本人通行凭证
被访人	确认或拒绝预约	本人作为被访人的预约和审批记录
部门审批人员	审批本部门预约	本部门预约、审批记录和统计数据
门岗安保人员	通行码核验、入校登记、离校登记、在校访客查询	通行凭证、出入校记录、校门数据
系统管理员	系统配置、用户权限、黑名单、日志维护	全量管理数据
校级管理人员	访客记录查询和统计报表	全校汇总数据和记录查询结果

2.3 功能需求

表 4: 核心功能需求表

编号	功能	需求说明
FR-01	访客预约申请	访客填写身份、事由、被访人、时间、车辆和随行人员信息，系统写入 visitor、visit_apply 等表。
FR-02	黑名单检查	提交预约和门岗核验时按手机号、证件号检查 blacklist 表，识别风险访客。
FR-03	被访人确认	被访人处理待确认预约，确认后进入部门审批，拒绝后流程终止。
FR-04	部门审批	部门审批人员处理本部门预约，审批通过后允许生成通行凭证。
FR-05	通行凭证	审批通过后生成 pass_code，记录通行码、有效期和使用状态。
FR-06	门岗核验	按预约编号、手机号、证件号或通行码核验访客是否允许入校。
FR-07	出入校登记	记录入校校门、离校校门、安保人员、时间和访问状态。
FR-08	超时未离校	识别计划离校时间已过但未离校的访客，并标记风险状态。
FR-09	统计报表	展示今日、本周、本月、部门排行、校门通行、趋势、审批通过率等指标。

FR-10	系统管理	维护用户、角色、权限、部门、校门、字典、日志和自动化记录。
-------	------	-------------------------------

2.4 性能需求

表 5: 性能需求表

类别	要求
响应时间	普通列表、详情和统计查询在演示数据规模下应在 2 秒内返回。
并发能力	满足课程演示和多角色同时操作场景，核心流程接口保持无明显阻塞。
可维护性	字段命名采用小写下划线，接口返回统一结构，前后端字段与数据库字段保持映射一致。
可扩展性	部门、校门、字典、权限等基础数据可扩展，不影响核心预约流程。

2.5 运行需求

表 6: 运行环境需求表

环境	要求
数据库	MySQL 8, InnoDB, utf8mb4 字符集。
后端	JDK 17、Spring Boot 3、MyBatis Plus、JWT、Swagger/OpenAPI。
前端	Node.js、Vue 3、Vite、Element Plus、Axios、Vue Router、ECharts。
自动化	Playwright 负责截图，XeLaTeX 负责编译最终报告。

2.6 安全需求

表 7: 安全需求表

安全项	要求
认证	登录成功返回 JWT Token，后续请求携带 Authorization 请求头。
密码	默认账号用于演示，后端密码字段按加密摘要存储。
授权	基于角色控制菜单和接口访问范围，越权访问返回明确错误。
审计	登录、预约、审批、出入校、黑名单维护和报表查询写入 operation_log。

风险控制

黑名单访客在预约或核验阶段被拦截或提示风险。

2.7 数据需求

系统必须保存访客信息、预约申请信息、审批记录、通行凭证、出入校记录、黑名单信息、用户角色权限、部门、校门、通知、操作日志、字典项、截图记录和报告记录等数据。所有关键业务数据都应设置主键，跨表关联通过外键或业务字段约束表达；状态字段应具有明确含义，并能支撑前端状态标签和统计报表。

2.8 数据流程图

数据流程图从数据流角度描述外部实体、处理过程和数据存储之间的关系。下列图示分别给出系统边界和核心业务处理过程，为后续概念结构设计和逻辑结构设计提供依据。

2.8.1 顶层数据流程图

顶层数据流程图展示访客、校内人员、审批人员、门岗和管理人员与系统之间的主要数据交换。

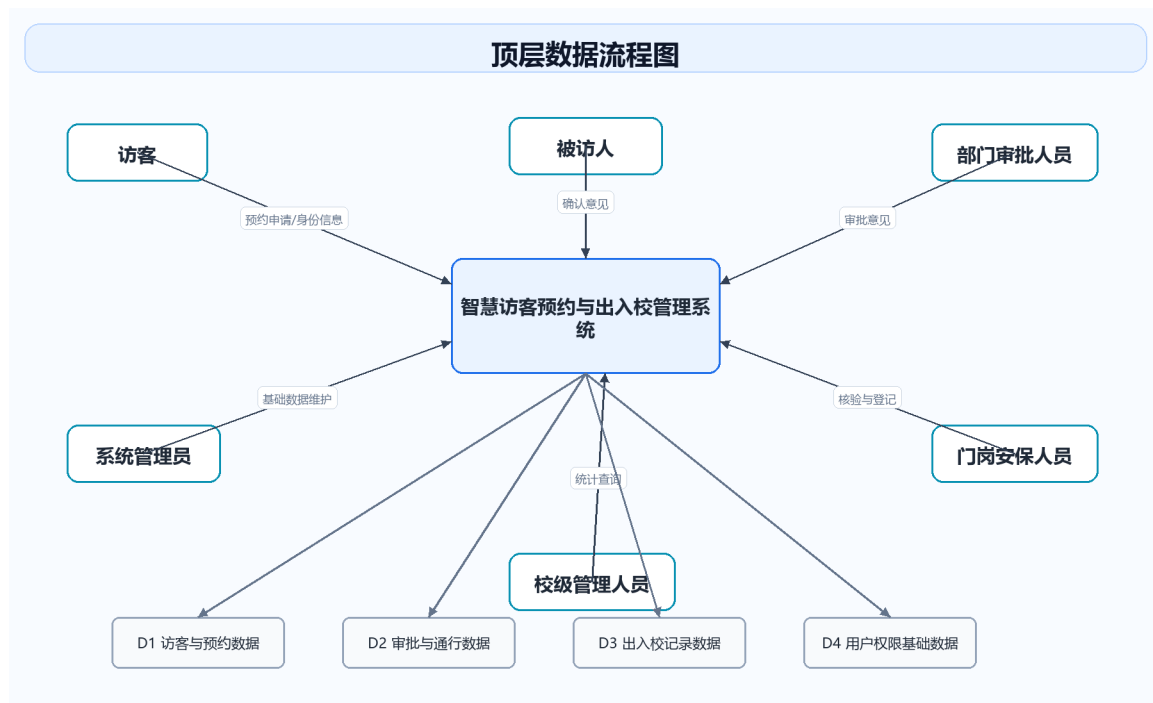


图 1：顶层数据流程图

2.8.2 二层数据流程图

二层数据流程图进一步展开预约审批、门岗核验、出入校登记和统计查询等处理过程。

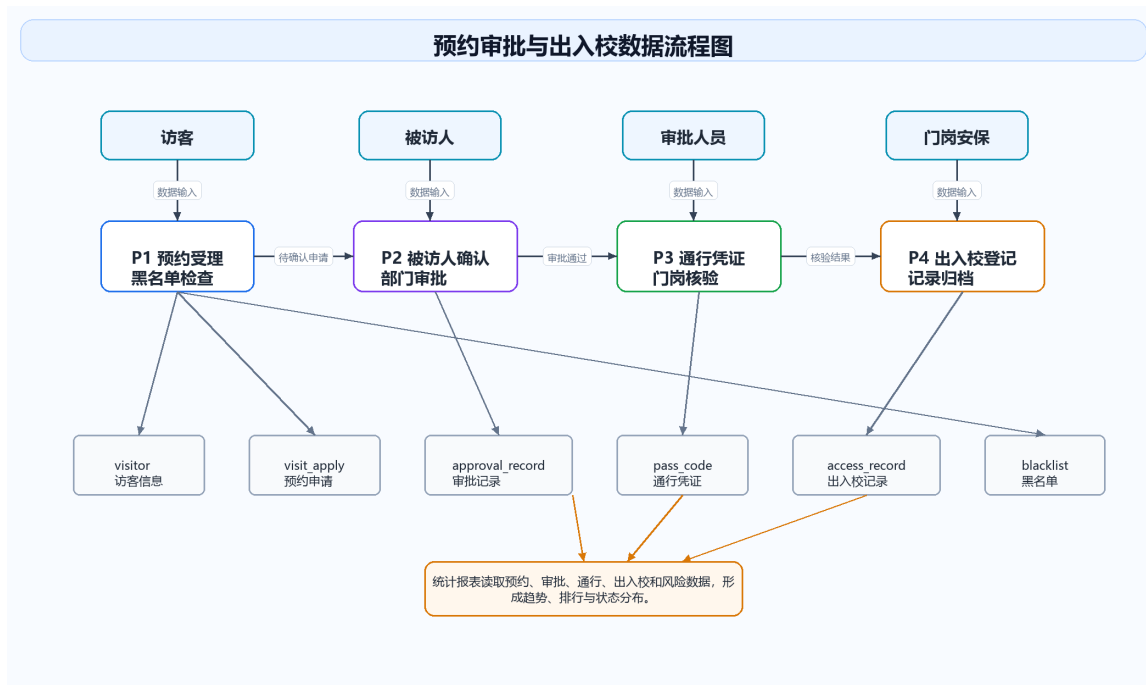


图 2: 二层数据流程图

2.9 数据字典

表 8: 核心数据字典摘要

类别	数据对象	说明
数据流	预约申请信息	访客提交的身份、来访事由、被访人、时间、车辆和随行人员等信息。
数据流	审批结果信息	被访人确认结果、部门审批结果、审批意见和审批时间。
数据存储	visit_apply	保存预约申请主记录，是核心业务主表。
数据存储	approval_record	保存被访人确认、部门审批等审批链路。
数据存储	access_record	保存入校、离校、异常和超时记录。
外部实体	访客	预约申请的发起人和通行凭证使用者。
外部实体	门岗安保人员	通行核验和出入校登记的执行者。
核心数据项	apply_status	预约状态，包含待确认、审批通过、审批拒绝、已取消等。

核心数据项	access_status	访问状态，包含未入校、已入校、已离校、超时未离校等。
核心数据项	pass_code	通行凭证编码，用于门岗核验。

三、 系统设计

3.1 本章概述

系统设计将需求分析结果转换为可实现的数据模型和系统结构。本章包括概念结构设计、E-R 图、逻辑结构设计、关系模式、系统功能模块图和其它设计图，保证后续 MySQL 表结构、后端 Entity、前端页面和报告描述一致。

3.2 概念结构设计说明

概念结构设计阶段从业务流程中抽取实体和联系。核心业务实体包括 visitor、visit_apply、approval_record、pass_code、access_record、blacklist、campus_gate 等；权限实体包括 sys_user、sys_role、sys_permission 以及用户角色、角色权限中间表；支撑实体包括 notice、operation_log、dict_type、dict_item、screenshot_record 和 report_record。设计时将“访客提交预约”作为主线，确保每个实体都能在核心流程中找到业务来源。

表 9: 主要实体及存在原因

实体	存在原因
visitor	保存访客身份基础信息，避免同一访客多次预约时重复录入。
visit_apply	保存预约申请主记录，连接访客、被访人、部门、车辆、审批和出入校流程。
approval_record	保存被访人确认、部门审批等多阶段审批过程。
pass_code	保存审批通过后的通行凭证和有效期。
access_record	保存门岗核验后的入校、离校和异常记录。
blacklist	保存限制或风险访客，支撑预约和核验阶段的安全检查。
sys_user/sys_role/sys_permission	支撑系统登录认证、角色授权和菜单接口控制。
operation_log	保存关键业务操作日志，支撑审计和测试说明。

3.3 E-R 图

为了避免一张图实体过多、线条交叉严重，报告将 E-R 图拆分为总体简化 E-R 图、核心业务 E-R 图、用户权限 E-R 图和系统支撑 E-R 图。正文重点展示总体关系和核心

业务关系，权限与支撑实体作为补充说明。

3.3.1 总体简化 E-R 图

总体简化 E-R 图只展示主要实体和主流程关系，用于报告正文快速说明系统数据模型全貌。

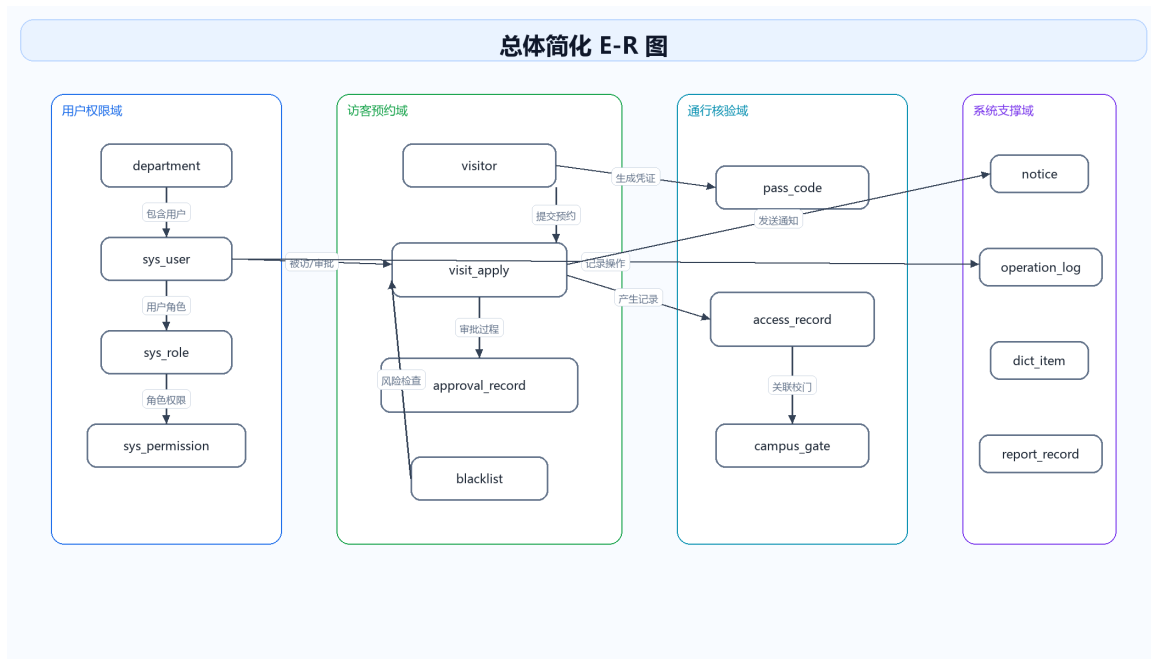


图 3: 总体简化 E-R 图

3.3.2 核心业务 E-R 图

核心业务 E-R 图围绕访客预约、审批、通行凭证、出入校记录和黑名单展开，是后续关系模式转换的主要依据。

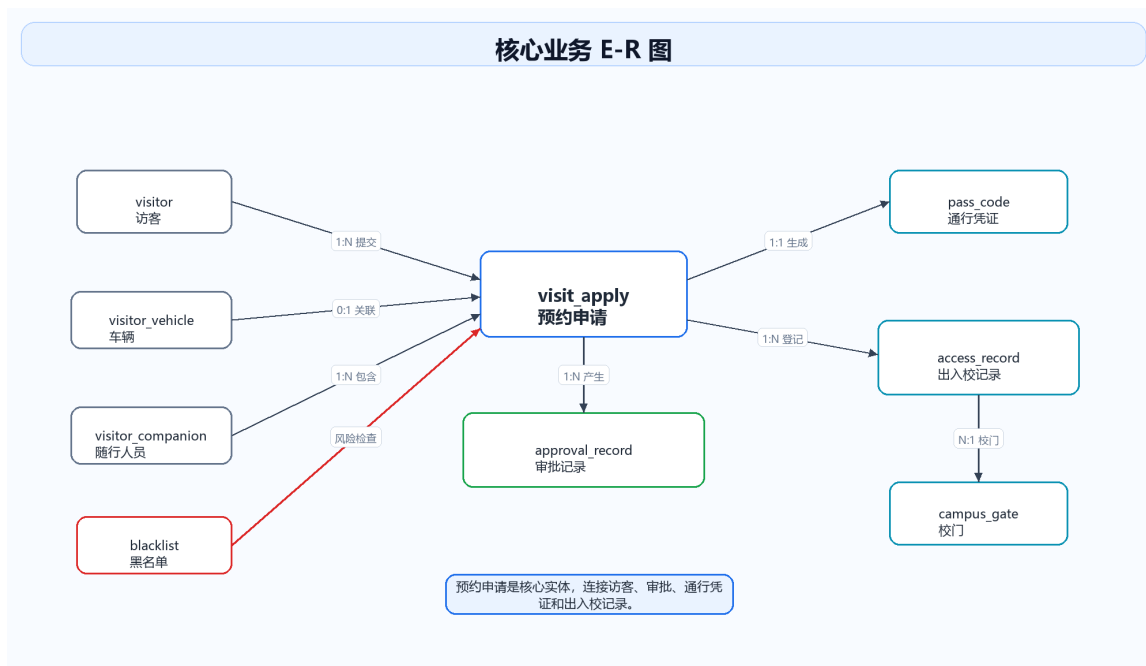


图 4: 核心业务 E-R 图

3.3.3 用户权限 E-R 图

用户权限 E-R 图展示系统用户、角色、权限、部门及两个多对多中间表之间的联系。

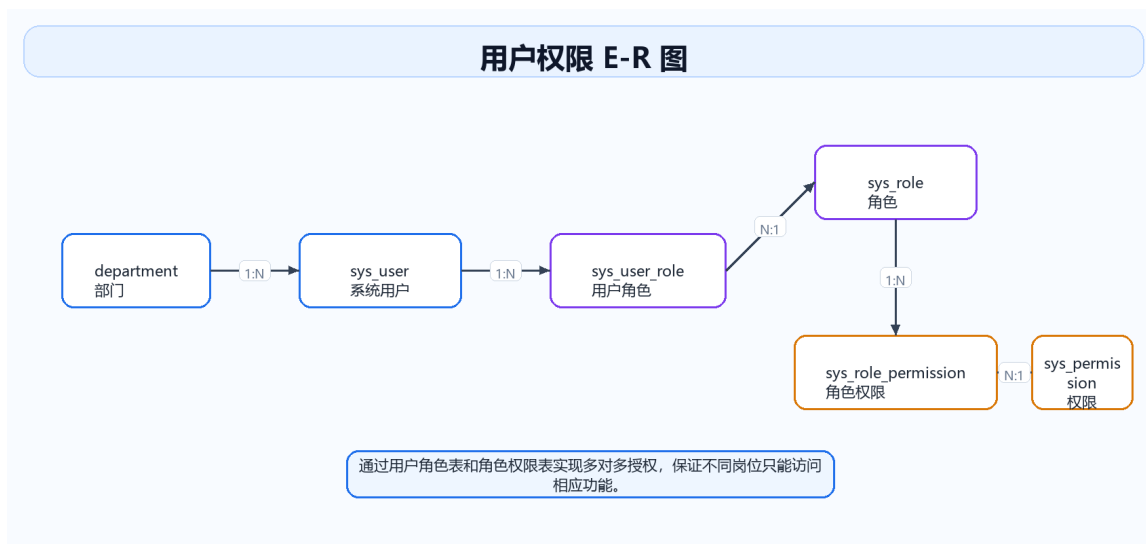


图 5: 用户权限 E-R 图

3.3.4 系统支撑 E-R 图

系统支撑 E-R 图展示通知、日志、字典、截图记录和报告记录等支撑性实体。

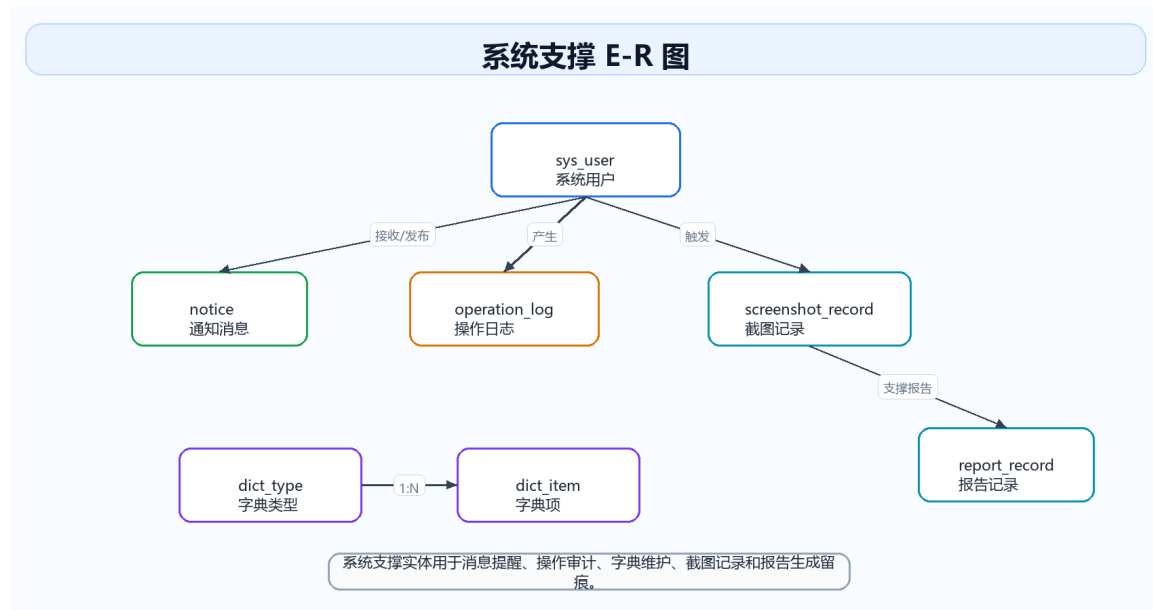


图 6: 系统支撑 E-R 图

3.4 逻辑结构设计说明

逻辑结构设计将概念实体转换为 MySQL 关系表。实体表统一使用 id 作为主键，时间字段统一使用 create_time、update_time，软删除字段统一使用 deleted。多对多关系通过 sys_user_role 和 sys_role_permission 中间表表达；一对多关系通过外键字段或业务关联字段表达；预约、审批、通行和出入校状态通过状态字段表达。

表 10: 逻辑表结构摘要

表名	关键字段摘要	说明
sys_user	id BIGINT; username VARCHAR(50); password_hash VARCHAR(255); real_name VARCHAR(50); phone VARCHAR(20); email VARCHAR(100); department_id BIGINT; user_type VARCHAR(30)	数据库物理表，与关系模式和后端 Entity 对应
sys_role	id BIGINT; role_code VARCHAR(50); role_name VARCHAR(50); role_desc VARCHAR(255); status VARCHAR(30); create_time DATETIME; update_time DATETIME; deleted TINYINT	数据库物理表，与关系模式和后端 Entity 对应

sys_permission	id BIGINT; permission_code VARCHAR(100); permission_name VARCHAR(100); permission_type VARCHAR(30); parent_id BIGINT; route_path VARCHAR(200); component_path VARCHAR(200); api_path VARCHAR(200)	数据库物理表, 与关系模式和后 端 Entity 对应
sys_user_role	id BIGINT; user_id BIGINT; role_id BIGINT; create_time DATETIME; update_time DATETIME; deleted TINYINT; PRIMARY KEY; UNIQUE KEY	数据库物理表, 与关系模式和后 端 Entity 对应
sys_role_permission	id BIGINT; role_id BIGINT; permission_id BIGINT; create_time DATETIME; update_time DATETIME; deleted TINYINT; PRIMARY KEY; UNIQUE KEY	数据库物理表, 与关系模式和后 端 Entity 对应
department	id BIGINT; parent_id BIGINT; dept_code VARCHAR(50); dept_name VARCHAR(100); leader_user_id BIGINT; phone VARCHAR(20); sort_order INT; status VARCHAR(30)	数据库物理表, 与关系模式和后 端 Entity 对应
campus_gate	id BIGINT; gate_code VARCHAR(50); gate_name VARCHAR(100); gate_location VARCHAR(200); gate_type VARCHAR(30); status VARCHAR(30); remark VARCHAR(255); create_time DATETIME	数据库物理表, 与关系模式和后 端 Entity 对应
visitor	id BIGINT; visitor_name VARCHAR(50); id_type VARCHAR(30); id_number VARCHAR(50); phone VARCHAR(20); company VARCHAR(100); gender VARCHAR(10); visitor_level VARCHAR(30)	数据库物理表, 与关系模式和后 端 Entity 对应
visitor_vehicle	id BIGINT; visitor_id BIGINT; plate_no VARCHAR(20); vehicle_type VARCHAR(30); color VARCHAR(30); brand VARCHAR(50); status VARCHAR(30); create_time DATETIME	数据库物理表, 与关系模式和后 端 Entity 对应

visitor_companion	id BIGINT; apply_id BIGINT; companion_name VARCHAR(50); id_type VARCHAR(30); id_number VARCHAR(50); phone VARCHAR(20); relation_remark VARCHAR(100); create_time DATETIME	数据库物理表, 与关系模式和后 端 Entity 对应
visit_apply	id BIGINT; apply_no VARCHAR(50); visitor_id BIGINT; host_user_id BIGINT; department_id BIGINT; vehicle_id BIGINT; visit_reason VARCHAR(255); plan_start_time DATETIME	数据库物理表, 与关系模式和后 端 Entity 对应
approval_record	id BIGINT; apply_id BIGINT; approval_step VARCHAR(30); approver_user_id BIGINT; approval_result VARCHAR(30); approval_comment VARCHAR(255); approval_time DATETIME; sort_order INT	数据库物理表, 与关系模式和后 端 Entity 对应
pass_code	id BIGINT; apply_id BIGINT; pass_code VARCHAR(64); qr_content VARCHAR(500); valid_from DATETIME; valid_to DATETIME; pass_status VARCHAR(30); used_time DATETIME	数据库物理表, 与关系模式和后 端 Entity 对应
access_record	id BIGINT; apply_id BIGINT; visitor_id BIGINT; pass_code_id BIGINT; entry_gate_id BIGINT; exit_gate_id BIGINT; entry_guard_id BIGINT; exit_guard_id BIGINT	数据库物理表, 与关系模式和后 端 Entity 对应
blacklist	id BIGINT; visitor_id BIGINT; id_number VARCHAR(50); phone VARCHAR(20); reason VARCHAR(255); level VARCHAR(30); start_time DATETIME; end_time DATETIME	数据库物理表, 与关系模式和后 端 Entity 对应
notice	id BIGINT; receiver_user_id BIGINT; receiver_type VARCHAR(30); title VARCHAR(100); content VARCHAR(1000); business_type VARCHAR(50); business_id BIGINT; read_status VARCHAR(30)	数据库物理表, 与关系模式和后 端 Entity 对应

operation_log	id BIGINT; operator_user_id BIGINT; operator_name VARCHAR(50); module_name VARCHAR(100); operation_type VARCHAR(50); request_method VARCHAR(10); request_url VARCHAR(255); operation_result VARCHAR(30)	数据库物理表, 与关系模式和后 端 Entity 对应
dict_type	id BIGINT; type_code VARCHAR(50); type_name VARCHAR(100); status VARCHAR(30); remark VARCHAR(255); create_time DATETIME; update_time DATETIME; deleted TINYINT	数据库物理表, 与关系模式和后 端 Entity 对应
dict_item	id BIGINT; type_id BIGINT; item_code VARCHAR(50); item_name VARCHAR(100); item_value VARCHAR(100); sort_order INT; status VARCHAR(30); remark VARCHAR(255)	数据库物理表, 与关系模式和后 端 Entity 对应
screenshot_record	id BIGINT; screenshot_code VARCHAR(50); page_name VARCHAR(100); route_path VARCHAR(200); role_code VARCHAR(50); file_path VARCHAR(255); description VARCHAR(255); capture_time DATETIME	数据库物理表, 与关系模式和后 端 Entity 对应
report_record	id BIGINT; report_code VARCHAR(50); report_name VARCHAR(100); markdown_path VARCHAR(255); word_path VARCHAR(255); generate_status VARCHAR(30); generate_time DATETIME; generated_by BIGINT	数据库物理表, 与关系模式和后 端 Entity 对应

3.5 关系模式说明

系统关系模式与 MySQL 表结构一一对应。核心关系模式包括 visitor(id, visitor_name, phone, id_number, ...)、visit_apply(id, apply_no, visitor_id, host_user_id, department_id, apply_status, access_status, ...)、approval_record(id, apply_id, approval_step, approval_result, ...)、pass_code(id, apply_id, code, valid_from, valid_to, status, ...)、access_record(id, apply_id, visitor_id, entry_gate_id, exit_gate_id, access_status, ...)。这些关系模式既能表达实体属性，也能表达核心业务联系。

3.6 系统功能模块图

系统功能模块图按角色和业务域划分为访客端、被访人端、审批管理、门岗管理、访客记录、安全管理、统计报表、系统管理和自动化支撑等模块。

3.6.1 系统功能模块图

模块图展示系统功能分解及各模块边界，便于说明前端菜单、后端接口和权限控制的对应关系。



图 7：系统功能模块图

3.7 其它设计图

3.7.1 访客预约审批流程图

访客预约审批流程图展示从预约提交到审批通过或拒绝的状态流转，并体现黑名单拦截、被访人拒绝和部门审批拒绝等异常分支。

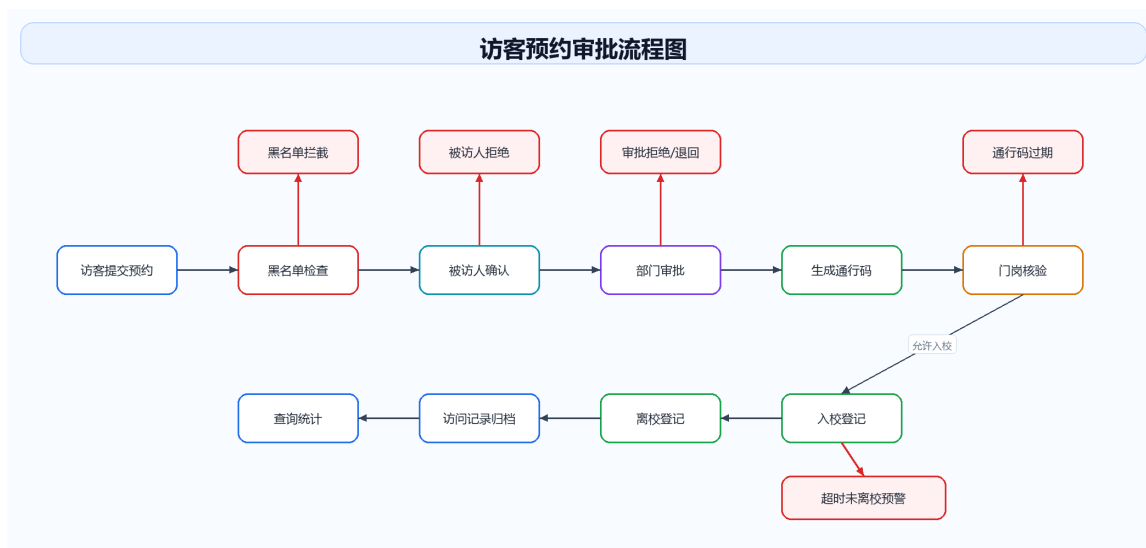


图 8: 访客预约审批流程图

3.7.2 门岗核验入校流程图

门岗核验流程图展示通行码核验、时间有效性检查、黑名单检查、入校登记、离校登记和超时未离校处理。

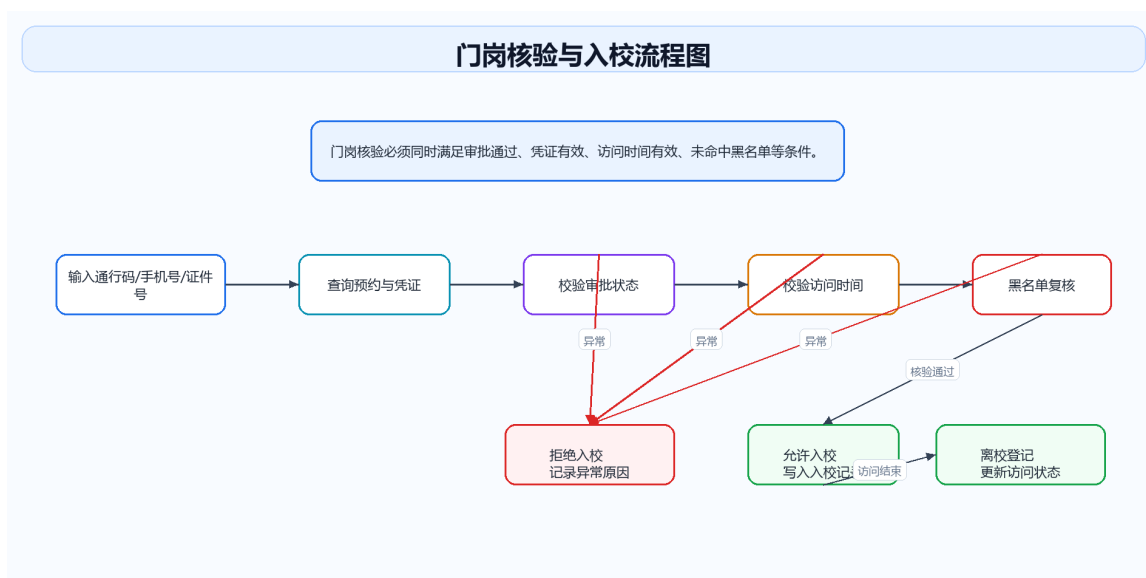


图 9: 门岗核验入校流程图

3.7.3 系统架构图

系统架构图展示浏览器前端、Spring Boot 后端、MySQL 数据库、Playwright 截图脚本和 LaTeX 报告生成脚本之间的关系。

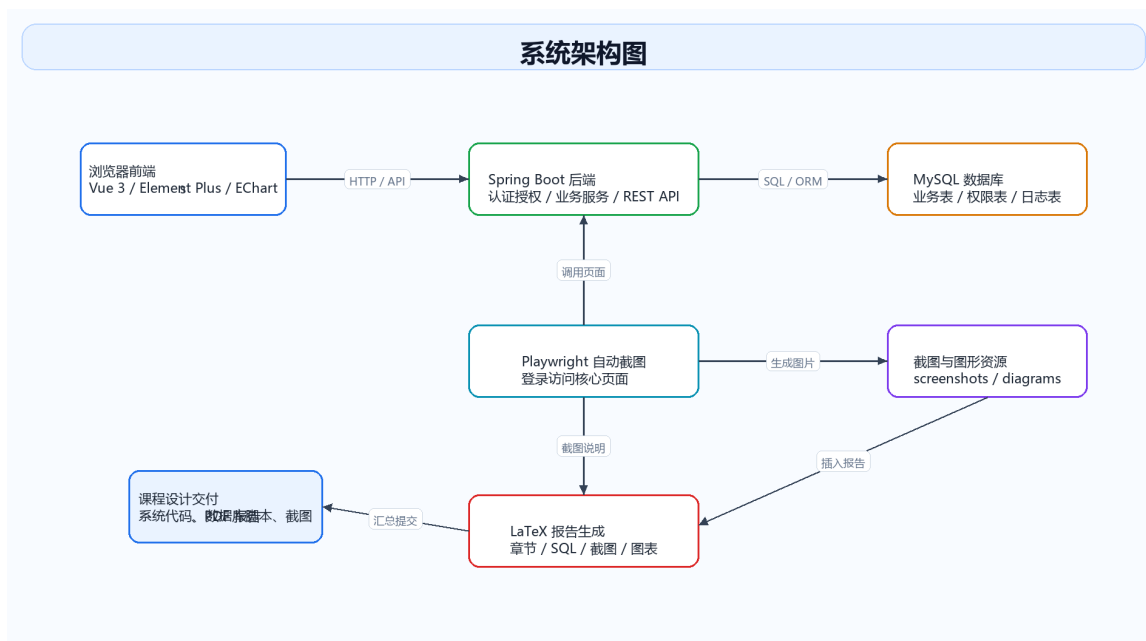


图 10: 系统架构图

3.7.4 数据库表关系图

数据库表关系图适合放入附录，用于展示主外键关系和表间依赖。

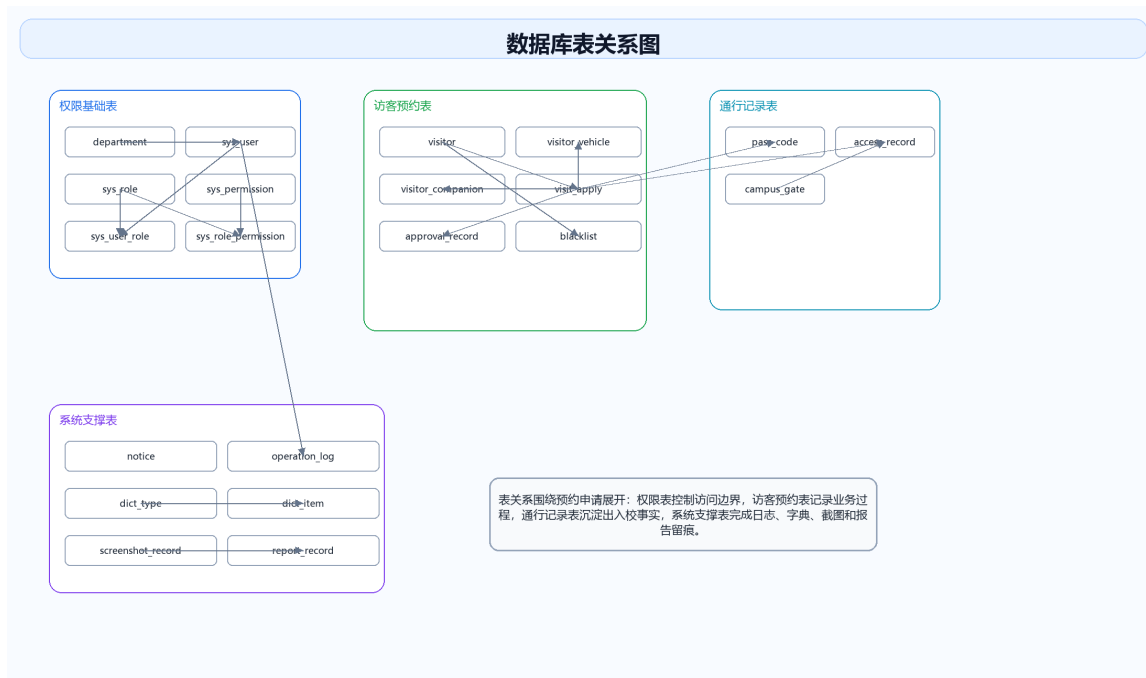


图 11: 数据库表关系图

3.7.5 自动截图与报告生成流程图

自动化支撑流程图展示演示数据、页面截图和报告生成之间的衔接关系，说明系统如何形成可复核的课程设计材料。

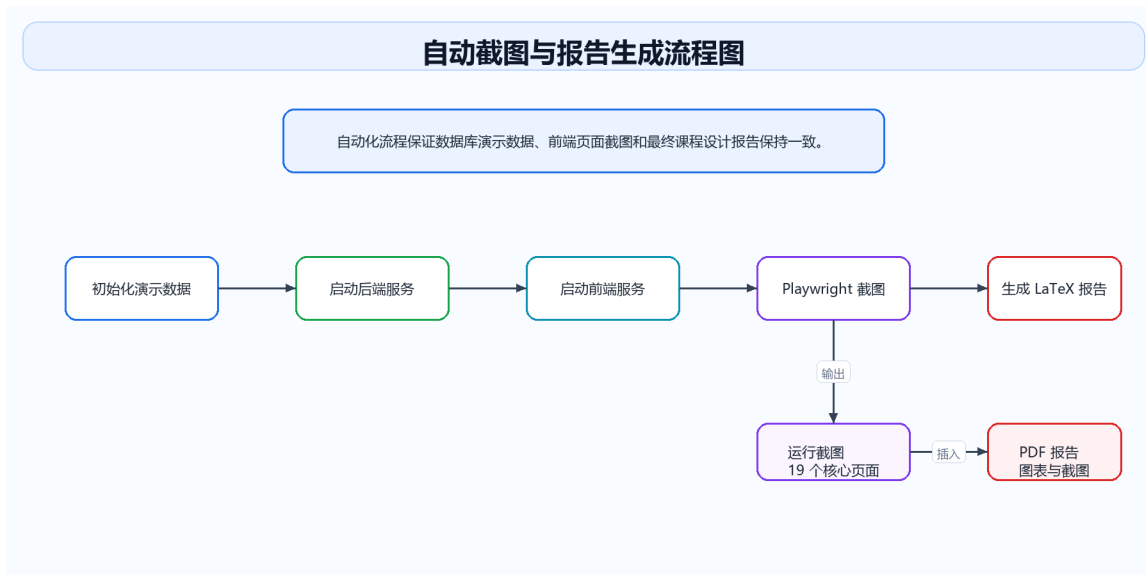


图 12: 自动截图与报告生成流程图

四、详细设计

4.1 本章概述

详细设计重点说明系统如何通过 SQL 支撑核心业务查询和统计分析。以下查询围绕访客预约、审批流转、门岗通行、风险控制和统计报表展开，体现关系数据库在数据关联、聚合统计和状态筛选方面的设计作用。

4.2 SQL-01 查询某访客的历史预约记录

查询名称：查询某访客的历史预约记录。

查询目的：用于访客记录查询页，按手机号回溯访客所有历史预约，关联展示被访部门、被访人、预约状态和访问状态。

涉及数据表：visit_apply, visitor, department, sys_user。

SQL 语句：

```
1 SELECT
2   v.visitor_name,
3   v.phone,
4   va.apply_no,
```

```
5   va.visit_reason,  
6   d.dept_name,  
7   u.real_name AS host_name,  
8   va.plan_start_time,  
9   va.plan_end_time,  
10  va.apply_status,  
11  va.access_status  
12 FROM visit_apply va  
13 JOIN visitor v ON va.visitor_id = v.id  
14 LEFT JOIN department d ON va.department_id = d.id  
15 LEFT JOIN sys_user u ON va.host_user_id = u.id  
16 WHERE va.deleted = 0 AND v.phone = '13950000001'  
17 ORDER BY va.plan_start_time DESC;
```

查询说明：该查询与系统页面、后端统计接口或课程设计报告中的业务说明保持一致，能够直接基于当前数据库脚本执行。

4.3 SQL-02 查询某被访人的待确认预约

查询名称：查询某被访人的待确认预约。

查询目的：用于被访人端待确认列表，保证被访人只处理与本人账号相关的待确认预约。

涉及数据表：visit_apply, visitor, sys_user。

SQL 语句：

```
1 SELECT va.apply_no, v.visitor_name, v.phone, va.visit_reason, va.plan_start_time, va.apply_status  
2 FROM visit_apply va  
3 JOIN visitor v ON va.visitor_id = v.id  
4 JOIN sys_user host ON va.host_user_id = host.id  
5 WHERE va.deleted = 0  
6       AND host.username = 'teacher01'  
7       AND va.apply_status = 'PENDING_HOST'  
8 ORDER BY va.plan_start_time ASC;
```

查询说明：该查询与系统页面、后端统计接口或课程设计报告中的业务说明保持一致，能够直接基于当前数据库脚本执行。

4.4 SQL-03 查询某部门的待审批预约

查询名称：查询某部门的待审批预约。

查询目的：用于部门审批页，筛选本部门已由被访人确认或进入部门审批阶段的预约申请。

涉及数据表：visit_apply, visitor, department, sys_user。

SQL 语句：

```
1 SELECT va.apply_no, d.dept_name, v.visitor_name, host.real_name AS host_name, va.visit_reason, va
   .plan_start_time, va.apply_status
2 FROM visit_apply va
3 JOIN visitor v ON va.visitor_id = v.id
4 JOIN department d ON va.department_id = d.id
5 LEFT JOIN sys_user host ON va.host_user_id = host.id
6 WHERE va.deleted = 0
7       AND d.dept_name = '信息与通信工程学院'
8       AND va.apply_status IN ('HOST_CONFIRMED', 'PENDING_DEPT')
9 ORDER BY va.plan_start_time ASC;
```

查询说明：该查询与系统页面、后端统计接口或课程设计报告中的业务说明保持一致，能够直接基于当前数据库脚本执行。

4.5 SQL-04 查询当天已入校访客

查询名称：查询当天已入校访客。

查询目的：用于门岗和统计页面，动态查询当天已经完成入校登记的访客。

涉及数据表：access_record, visit_apply, visitor, campus_gate, sys_user。

SQL 语句：

```
1 SELECT ar.id, va.apply_no, v.visitor_name, v.phone, g.gate_name AS entry_gate, guard.real_name AS
   guard_name, ar.entry_time
2 FROM access_record ar
3 JOIN visit_apply va ON ar.apply_id = va.id
4 JOIN visitor v ON ar.visitor_id = v.id
5 LEFT JOIN campus_gate g ON ar.entry_gate_id = g.id
6 LEFT JOIN sys_user guard ON ar.entry_guard_id = guard.id
7 WHERE ar.deleted = 0
8       AND ar.entry_time >= CURRENT_DATE()
9       AND ar.entry_time < DATE_ADD(CURRENT_DATE(), INTERVAL 1 DAY)
10 ORDER BY ar.entry_time DESC;
```

查询说明：该查询与系统页面、后端统计接口或课程设计报告中的业务说明保持一致，能够直接基于当前数据库脚本执行。

4.6 SQL-05 查询当天已入校但未离校访客

查询名称：查询当天已入校但未离校访客。

查询目的：用于当前在校访客页，查询当天已入校但尚未离校的访客记录。

涉及数据表：access_record, visit_apply, visitor, department。

SQL 语句：

```
1 SELECT ar.id, va.apply_no, v.visitor_name, v.phone, d.dept_name, ar.entry_time, ar.access_status
```

```
2 FROM access_record ar
3 JOIN visit_apply va ON ar.apply_id = va.id
4 JOIN visitor v ON ar.visitor_id = v.id
5 LEFT JOIN department d ON va.department_id = d.id
6 WHERE ar.deleted = 0
7     AND ar.access_status = 'ENTERED'
8     AND ar.entry_time >= CURRENT_DATE()
9     AND ar.entry_time < DATE_ADD(CURRENT_DATE(), INTERVAL 1 DAY)
10 ORDER BY ar.entry_time DESC;
```

查询说明：该查询与系统页面、后端统计接口或课程设计报告中的业务说明保持一致，能够直接基于当前数据库脚本执行。

4.7 SQL-06 查询超时未离校访客

查询名称：查询超时未离校访客。

查询目的：用于超时未离校页面，结合访问状态与计划离校时间识别风险访客。

涉及数据表：access_record, visit_apply, visitor。

SQL 语句：

```
1 SELECT va.apply_no, v.visitor_name, v.phone, va.plan_end_time, ar.entry_time, ar.overtime_flag,
   ar.abnormal_reason
2 FROM access_record ar
3 JOIN visit_apply va ON ar.apply_id = va.id
4 JOIN visitor v ON ar.visitor_id = v.id
5 WHERE ar.deleted = 0
6     AND (ar.access_status = 'OVERTIME' OR (ar.exit_time IS NULL AND va.plan_end_time < NOW()))
7 ORDER BY va.plan_end_time ASC;
```

查询说明：该查询与系统页面、后端统计接口或课程设计报告中的业务说明保持一致，能够直接基于当前数据库脚本执行。

4.8 SQL-07 按部门统计本月访客数量

查询名称：按部门统计本月访客数量。

查询目的：用于部门访客排行图表，按本月访问申请统计各部门接待量。

涉及数据表：visit_apply, department。

SQL 语句：

```
1 SELECT d.dept_name, COUNT(*) AS visitor_count
2 FROM visit_apply va
3 JOIN department d ON va.department_id = d.id
4 WHERE va.deleted = 0
5     AND va.plan_start_time >= DATE_FORMAT(CURRENT_DATE(), '%Y-%m-01')
6     AND va.plan_start_time < DATE_ADD(DATE_FORMAT(CURRENT_DATE(), '%Y-%m-01'), INTERVAL 1 MONTH)
```

```
7 GROUP BY d.id, d.dept_name
8 ORDER BY visitor_count DESC;
```

查询说明：该查询与系统页面、后端统计接口或课程设计报告中的业务说明保持一致，能够直接基于当前数据库脚本执行。

4.9 SQL-08 按校门统计本月通行数量

查询名称：按校门统计本月通行数量。

查询目的：用于校门通行统计图表，分别统计本月各校门入校、离校和总通行次数。

涉及数据表：campus_gate, access_record。

SQL 语句：

```
1 SELECT g.gate_name,
2        SUM(CASE WHEN ar.entry_gate_id = g.id THEN 1 ELSE 0 END) AS entry_count,
3        SUM(CASE WHEN ar.exit_gate_id = g.id THEN 1 ELSE 0 END) AS exit_count,
4        SUM(CASE WHEN ar.entry_gate_id = g.id THEN 1 ELSE 0 END) + SUM(CASE WHEN ar.exit_gate_id =
5        g.id THEN 1 ELSE 0 END) AS total_count
6 FROM campus_gate g
7 LEFT JOIN access_record ar ON ar.deleted = 0
8 AND (ar.entry_gate_id = g.id OR ar.exit_gate_id = g.id)
9 AND COALESCE(ar.entry_time, ar.exit_time) >= DATE_FORMAT(CURRENT_DATE(), '%Y-%m-01')
10 AND COALESCE(ar.entry_time, ar.exit_time) < DATE_ADD(DATE_FORMAT(CURRENT_DATE(), '%Y-%m-01'),
11 INTERVAL 1 MONTH)
12 WHERE g.deleted = 0
13 GROUP BY g.id, g.gate_name
14 ORDER BY total_count DESC;
```

查询说明：该查询与系统页面、后端统计接口或课程设计报告中的业务说明保持一致，能够直接基于当前数据库脚本执行。

4.10 SQL-09 查询黑名单访客

查询名称：查询黑名单访客。

查询目的：用于黑名单管理和预约风险检查，展示仍处于生效状态的限制入校人员。

涉及数据表：blacklist, visitor。

SQL 语句：

```
1 SELECT b.id, v.visitor_name, b.phone, b.id_number, b.reason, b.level, b.status, b.start_time, b.
2        end_time
3 FROM blacklist b
4 LEFT JOIN visitor v ON b.visitor_id = v.id
5 WHERE b.deleted = 0 AND b.status = 'ACTIVE'
6 ORDER BY b.level DESC, b.start_time DESC;
```

查询说明：该查询与系统页面、后端统计接口或课程设计报告中的业务说明保持一致，能够直接基于当前数据库脚本执行。

4.11 SQL-10 查询某预约的完整审批记录

查询名称：查询某预约的完整审批记录。

查询目的：用于预约详情页，按审批顺序展示某个预约的完整审批链路。

涉及数据表：approval_record, visit_apply, sys_user。

SQL 语句：

```
1 SELECT va.apply_no, ar.approval_step, approver.real_name AS approver_name, ar.approval_result, ar
   .approval_comment, ar.approval_time
2 FROM approval_record ar
3 JOIN visit_apply va ON ar.apply_id = va.id
4 LEFT JOIN sys_user approver ON ar.approver_user_id = approver.id
5 WHERE ar.deleted = 0 AND va.apply_no = (SELECT apply_no FROM visit_apply ORDER BY id LIMIT 1)
6 ORDER BY ar.sort_order ASC, ar.approval_time ASC;
```

查询说明：该查询与系统页面、后端统计接口或课程设计报告中的业务说明保持一致，能够直接基于当前数据库脚本执行。

4.12 SQL-11 查询某访客的车辆和随行人员

查询名称：查询某访客的车辆和随行人员。

查询目的：用于预约详情页，联合展示车辆和随行人员信息。

涉及数据表：visit_apply, visitor, visitor_vehicle, visitor_companion。

SQL 语句：

```
1 SELECT va.apply_no, v.visitor_name, vv.plate_no, vv.vehicle_type, vv.color, vc.companion_name, vc
   .phone AS companion_phone, vc.relation_remark
2 FROM visit_apply va
3 JOIN visitor v ON va.visitor_id = v.id
4 LEFT JOIN visitor_vehicle vv ON va.vehicle_id = vv.id
5 LEFT JOIN visitor_companion vc ON vc.apply_id = va.id AND vc.deleted = 0
6 WHERE va.deleted = 0 AND va.id = 2;
```

查询说明：该查询与系统页面、后端统计接口或课程设计报告中的业务说明保持一致，能够直接基于当前数据库脚本执行。

4.13 SQL-13 查询今日访客概览

查询名称：查询今日访客概览。

查询目的：用于首页驾驶舱，汇总今日访客、当前在校、超时未离校、待审批和黑名单风险。

涉及数据表：visit_apply, access_record, blacklist。

SQL 语句：

```

1 SELECT
2   (SELECT COUNT(*) FROM visit_apply WHERE deleted = 0 AND plan_start_time >= CURRENT_DATE() AND
    plan_start_time < DATE_ADD(CURRENT_DATE(), INTERVAL 1 DAY)) AS today_visitors,
3   (SELECT COUNT(*) FROM access_record WHERE deleted = 0 AND access_status = 'ENTERED') AS
    current_visitors,
4   (SELECT COUNT(*) FROM access_record WHERE deleted = 0 AND access_status = 'OVERTIME') AS
    overtime_visitors,
5   (SELECT COUNT(*) FROM visit_apply WHERE deleted = 0 AND apply_status IN ('PENDING_HOST', '
    HOST_CONFIRMED', 'PENDING_DEPT')) AS pending_approvals,
6   (SELECT COUNT(*) FROM blacklist WHERE deleted = 0 AND status = 'ACTIVE') AS blacklist_risks;
```

查询说明：该查询与系统页面、后端统计接口或课程设计报告中的业务说明保持一致，能够直接基于当前数据库脚本执行。

4.14 SQL-14 查询近七天访客趋势

查询名称：查询近七天访客趋势。

查询目的：用于近七天访客趋势图，使用递归日期表保证没有访问的日期也能显示为 0。

涉及数据表：days, visit_apply。

SQL 语句：

```

1 WITH RECURSIVE days AS (
2   SELECT DATE_SUB(CURRENT_DATE(), INTERVAL 6 DAY) AS stat_date
3   UNION ALL
4   SELECT DATE_ADD(stat_date, INTERVAL 1 DAY) FROM days WHERE stat_date < CURRENT_DATE()
5 )
6 SELECT DATE_FORMAT(days.stat_date, '%m-%d') AS stat_date,
7        COUNT(va.id) AS visitor_count
8 FROM days
9 LEFT JOIN visit_apply va ON va.deleted = 0
10    AND va.plan_start_time >= days.stat_date
11    AND va.plan_start_time < DATE_ADD(days.stat_date, INTERVAL 1 DAY)
12 GROUP BY days.stat_date
13 ORDER BY days.stat_date;
```

查询说明：该查询与系统页面、后端统计接口或课程设计报告中的业务说明保持一致，能够直接基于当前数据库脚本执行。

4.15 SQL-15 查询审批通过率

查询名称：查询审批通过率。

查询目的：用于审批通过率统计，计算部门审批通过、拒绝/退回、待处理数量和通过率。

涉及数据表：approval_record。

SQL 语句：

```

1 SELECT
2     SUM(CASE WHEN approval_result = 'APPROVED' THEN 1 ELSE 0 END) AS approved_count,
3     SUM(CASE WHEN approval_result IN ('REJECTED','RETURNED') THEN 1 ELSE 0 END) AS rejected_count,
4     SUM(CASE WHEN approval_result = 'PENDING' THEN 1 ELSE 0 END) AS pending_count,
5     ROUND(SUM(CASE WHEN approval_result = 'APPROVED' THEN 1 ELSE 0 END) * 100.0 / COUNT(*), 2) AS
        pass_rate
6 FROM approval_record
7 WHERE deleted = 0 AND approval_step = 'DEPT_APPROVAL';

```

查询说明：该查询与系统页面、后端统计接口或课程设计报告中的业务说明保持一致，能够直接基于当前数据库脚本执行。

4.16 SQL-18 访问事由分布

查询名称：访问事由分布。

查询目的：用于访问事由分布图，通过关键词归类展示家长、企业合作、后勤维修、学术交流等访问目的。

涉及数据表：visit_apply。

SQL 语句：

```

1 SELECT reason_type, COUNT(*) AS reason_count
2 FROM (
3     SELECT CASE
4         WHEN visit_reason LIKE '%家长%' OR visit_reason LIKE '%学籍%' THEN '学生家长'
5         WHEN visit_reason LIKE '%企业%' OR visit_reason LIKE '%合作%' THEN '企业合作'
6         WHEN visit_reason LIKE '%维修%' OR visit_reason LIKE '%后勤%' THEN '后勤维修'
7         WHEN visit_reason LIKE '%学术%' OR visit_reason LIKE '%研讨%' THEN '学术交流'
8         WHEN visit_reason LIKE '%面试%' OR visit_reason LIKE '%就业%' THEN '面试招聘'
9         WHEN visit_reason LIKE '%快递%' OR visit_reason LIKE '%配送%' THEN '快递配送'
10        WHEN visit_reason LIKE '%校友%' THEN '校友返校'
11        WHEN visit_reason LIKE '%专家%' OR visit_reason LIKE '%评审%' OR visit_reason LIKE '%讲座%'
12            THEN '外聘专家'
13        ELSE '其他'
14    END AS reason_type
15 FROM visit_apply
16 WHERE deleted = 0
17        AND plan_start_time >= DATE_FORMAT(CURRENT_DATE(), '%Y-%m-01')
18        AND plan_start_time < DATE_ADD(DATE_FORMAT(CURRENT_DATE(), '%Y-%m-01'), INTERVAL 1 MONTH)

```

```
18 ) t
19 GROUP BY reason_type
20 ORDER BY reason_count DESC;
```

查询说明：该查询与系统页面、后端统计接口或课程设计报告中的业务说明保持一致，能够直接基于当前数据库脚本执行。

4.17 SQL-19 审批状态分布

查询名称：审批状态分布。

查询目的：用于审批状态分布图，展示预约在待确认、审批通过、审批拒绝、取消等状态上的数量。

涉及数据表：visit_apply。

SQL 语句：

```
1 SELECT apply_status, COUNT(*) AS status_count
2 FROM visit_apply
3 WHERE deleted = 0
4 GROUP BY apply_status
5 ORDER BY status_count DESC;
```

查询说明：该查询与系统页面、后端统计接口或课程设计报告中的业务说明保持一致，能够直接基于当前数据库脚本执行。

4.18 SQL-20 入校离校状态分布

查询名称：入校离校状态分布。

查询目的：用于入校离校状态分布图，统计未入校、已入校、已离校、超时未离校等访问状态。

涉及数据表：visit_apply。

SQL 语句：

```
1 SELECT access_status, COUNT(*) AS status_count
2 FROM visit_apply
3 WHERE deleted = 0
4 GROUP BY access_status
5 ORDER BY status_count DESC;
```

查询说明：该查询与系统页面、后端统计接口或课程设计报告中的业务说明保持一致，能够直接基于当前数据库脚本执行。

4.19 SQL-21 黑名单风险数量

查询名称：黑名单风险数量。

查询目的：用于黑名单风险分析，按风险级别和状态统计风险数量。

涉及数据表：blacklist。

SQL 语句：

```
1 SELECT level, status, COUNT(*) AS risk_count
2 FROM blacklist
3 WHERE deleted = 0
4 GROUP BY level, status
5 ORDER BY risk_count DESC;
```

查询说明：该查询与系统页面、后端统计接口或课程设计报告中的业务说明保持一致，能够直接基于当前数据库脚本执行。

4.20 SQL-22 最近操作日志

查询名称：最近操作日志。

查询目的：用于系统日志页，展示最近关键操作记录，支撑系统审计。

涉及数据表：operation_log。

SQL 语句：

```
1 SELECT operator_name, module_name, operation_type, operation_result, request_url, operation_time
2 FROM operation_log
3 WHERE deleted = 0
4 ORDER BY operation_time DESC
5 LIMIT 10;
```

查询说明：该查询与系统页面、后端统计接口或课程设计报告中的业务说明保持一致，能够直接基于当前数据库脚本执行。

4.21 建表 SQL 摘要

数据库建表设计在主键、外键、索引、状态字段和时间字段上保持统一规范。以下摘要展示核心建表语句的结构特征，用于说明物理实现与逻辑结构设计之间的对应关系。

```
1 -- 重庆邮电大学智慧访客预约与出入校管理系统
2 -- MySQL 8 schema script
3
4 USE cqupt_visitor_system;
5 SET NAMES utf8mb4;
6 SET FOREIGN_KEY_CHECKS = 0;
7
```

```

8 DROP TABLE IF EXISTS report_record;
9 DROP TABLE IF EXISTS screenshot_record;
10 DROP TABLE IF EXISTS dict_item;
11 DROP TABLE IF EXISTS operation_log;
12 DROP TABLE IF EXISTS notice;
13 DROP TABLE IF EXISTS blacklist;
14 DROP TABLE IF EXISTS access_record;
15 DROP TABLE IF EXISTS pass_code;
16 DROP TABLE IF EXISTS approval_record;
17 DROP TABLE IF EXISTS visitor_companion;
18 DROP TABLE IF EXISTS visit_apply;
19 DROP TABLE IF EXISTS visitor_vehicle;
20 DROP TABLE IF EXISTS sys_role_permission;
21 DROP TABLE IF EXISTS sys_user_role;
22 DROP TABLE IF EXISTS sys_user;
23 DROP TABLE IF EXISTS dict_type;
24 DROP TABLE IF EXISTS visitor;
25 DROP TABLE IF EXISTS campus_gate;
26 DROP TABLE IF EXISTS sys_permission;
27 DROP TABLE IF EXISTS sys_role;
28 DROP TABLE IF EXISTS department;
29
30 SET FOREIGN_KEY_CHECKS = 1;
31
32 CREATE TABLE department (
33     id BIGINT NOT NULL AUTO_INCREMENT COMMENT '部门主键',
34     parent_id BIGINT NULL COMMENT '上级部门',
35     dept_code VARCHAR(50) NOT NULL COMMENT '部门编码',
36     dept_name VARCHAR(100) NOT NULL COMMENT '部门名称',
37     leader_user_id BIGINT NULL COMMENT '部门负责人用户编号',
38     phone VARCHAR(20) NULL COMMENT '联系电话',
39     sort_order INT NOT NULL DEFAULT 0 COMMENT '排序号',
40     status VARCHAR(30) NOT NULL DEFAULT 'ENABLED' COMMENT '部门状态: ENABLED、DISABLED',
41     create_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
42     update_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP COMMENT '更新时间',
43     deleted TINYINT NOT NULL DEFAULT 0 COMMENT '软删除标记: 0未删除, 1已删除',
44     PRIMARY KEY (id),
45     UNIQUE KEY uk_department_code (dept_code),
46     KEY idx_department_parent (parent_id),
47     KEY idx_department_leader (leader_user_id),
48     KEY idx_department_status (status),
49     CONSTRAINT fk_department_parent FOREIGN KEY (parent_id) REFERENCES department(id) ON DELETE SET NULL
50 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='部门表';
51
52 CREATE TABLE sys_role (
53     id BIGINT NOT NULL AUTO_INCREMENT COMMENT '角色主键',
54     role_code VARCHAR(50) NOT NULL COMMENT '角色编码',
55     role_name VARCHAR(50) NOT NULL COMMENT '角色名称',
56     role_desc VARCHAR(255) NULL COMMENT '角色说明',

```

```

57 status VARCHAR(30) NOT NULL DEFAULT 'ENABLED' COMMENT '角色状态: ENABLED、DISABLED',
58 create_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
59 update_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP COMMENT '更
    新时间',
60 deleted TINYINT NOT NULL DEFAULT 0 COMMENT '软删除标记',
61 PRIMARY KEY (id),
62 UNIQUE KEY uk_sys_role_code (role_code),
63 KEY idx_sys_role_status (status)
64 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='角色表';
65
66 CREATE TABLE sys_permission (
67     id BIGINT NOT NULL AUTO_INCREMENT COMMENT '权限主键',
68     permission_code VARCHAR(100) NOT NULL COMMENT '权限编码',
69     permission_name VARCHAR(100) NOT NULL COMMENT '权限名称',
70     permission_type VARCHAR(30) NOT NULL DEFAULT 'MENU' COMMENT '权限类型: MENU、BUTTON、API',
71     parent_id BIGINT NULL COMMENT '父级权限',
72     route_path VARCHAR(200) NULL COMMENT '前端路由',
73     component_path VARCHAR(200) NULL COMMENT '前端组件路径',
74     api_path VARCHAR(200) NULL COMMENT '后端接口路径',
75     sort_order INT NOT NULL DEFAULT 0 COMMENT '排序号',
76     status VARCHAR(30) NOT NULL DEFAULT 'ENABLED' COMMENT '权限状态',
77     create_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
78     update_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP COMMENT '更
    新时间',
79     deleted TINYINT NOT NULL DEFAULT 0 COMMENT '软删除标记',
80     PRIMARY KEY (id),
81     UNIQUE KEY uk_sys_permission_code (permission_code),
82     KEY idx_sys_permission_parent (parent_id),
83     KEY idx_sys_permission_type (permission_type),
84     CONSTRAINT fk_permission_parent FOREIGN KEY (parent_id) REFERENCES sys_permission(id) ON DELETE
        SET NULL
85 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='权限表';
86
87 CREATE TABLE campus_gate (
88     id BIGINT NOT NULL AUTO_INCREMENT COMMENT '校门主键',
89     gate_code VARCHAR(50) NOT NULL COMMENT '校门编码',
90     gate_name VARCHAR(100) NOT NULL COMMENT '校门名称',
91     gate_location VARCHAR(200) NULL COMMENT '校门位置',
92     gate_type VARCHAR(30) NOT NULL DEFAULT 'NORMAL' COMMENT '校门类型: NORMAL、VEHICLE、PEDESTRIAN'
        ,
93     status VARCHAR(30) NOT NULL DEFAULT 'ENABLED' COMMENT '校门状态',
94     remark VARCHAR(255) NULL COMMENT '备注',
95     create_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
96     update_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP COMMENT '更
    新时间',
97     deleted TINYINT NOT NULL DEFAULT 0 COMMENT '软删除标记',
98     PRIMARY KEY (id),
99     UNIQUE KEY uk_campus_gate_code (gate_code),
100     KEY idx_campus_gate_status (status)
101 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='校门表';
102

```

```

103 CREATE TABLE visitor (
104     id BIGINT NOT NULL AUTO_INCREMENT COMMENT '访客主键',
105     visitor_name VARCHAR(50) NOT NULL COMMENT '访客姓名',
106     id_type VARCHAR(30) NOT NULL DEFAULT 'ID_CARD' COMMENT '证件类型',
107     id_number VARCHAR(50) NOT NULL COMMENT '证件号码',
108     phone VARCHAR(20) NOT NULL COMMENT '手机号',
109     company VARCHAR(100) NULL COMMENT '单位',
110     gender VARCHAR(10) NULL COMMENT '性别',
111     visitor_level VARCHAR(30) NOT NULL DEFAULT 'NORMAL' COMMENT '访客等级: NORMAL、VIP、RISK',
112     status VARCHAR(30) NOT NULL DEFAULT 'NORMAL' COMMENT '访客状态',
113     create_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
114     update_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP COMMENT '更新时间',
115     deleted TINYINT NOT NULL DEFAULT 0 COMMENT '软删除标记',
116     PRIMARY KEY (id),
117     UNIQUE KEY uk_visitor_id_number (id_type, id_number),
118     KEY idx_visitor_phone (phone),
119     KEY idx_visitor_status (status)
120 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='访客表';
121
122 CREATE TABLE dict_type (
123     id BIGINT NOT NULL AUTO_INCREMENT COMMENT '字典类型主键',
124     type_code VARCHAR(50) NOT NULL COMMENT '字典类型编码',
125     type_name VARCHAR(100) NOT NULL COMMENT '字典类型名称',
126     status VARCHAR(30) NOT NULL DEFAULT 'ENABLED' COMMENT '状态',
127     remark VARCHAR(255) NULL COMMENT '备注',
128     create_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
129     update_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP COMMENT '更新时间',
130     deleted TINYINT NOT NULL DEFAULT 0 COMMENT '软删除标记',
131     PRIMARY KEY (id),
132     UNIQUE KEY uk_dict_type_code (type_code),
133     KEY idx_dict_type_status (status)
134 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='字典类型表';
135
136 CREATE TABLE sys_user (
137     id BIGINT NOT NULL AUTO_INCREMENT COMMENT '用户主键',
138     username VARCHAR(50) NOT NULL COMMENT '登录账号',
139     password_hash VARCHAR(255) NOT NULL COMMENT '加密后的密码',
140     real_name VARCHAR(50) NOT NULL COMMENT '真实姓名',
141     phone VARCHAR(20) NOT NULL COMMENT '手机号',
142     email VARCHAR(100) NULL COMMENT '邮箱',
143     department_id BIGINT NULL COMMENT '所属部门',
144     user_type VARCHAR(30) NOT NULL COMMENT '用户类型: HOST、APPROVER、GUARD、ADMIN、MANAGER',
145     status VARCHAR(30) NOT NULL DEFAULT 'ENABLED' COMMENT '账号状态',
146     last_login_time DATETIME NULL COMMENT '最近登录时间',
147     create_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
148     update_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP COMMENT '更新时间',
149     deleted TINYINT NOT NULL DEFAULT 0 COMMENT '软删除标记',
150     PRIMARY KEY (id),

```

```
151  UNIQUE KEY uk_sys_user_username (username),
152  KEY idx_sys_user_phone (phone),
153  KEY idx_sys_user_department (department_id),
154  KEY idx_sys_user_status (status),
155  CONSTRAINT fk_user_department FOREIGN KEY (department_id) REFERENCES department(id) ON DELETE
      SET NULL
156 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='系统用户表';
157
158 ALTER TABLE department
159   ADD CONSTRAINT fk_department_leader FOREIGN KEY (leader_user_id) REFERENCES sys_user(id) ON
      DELETE SET NULL;
160
161 CREATE TABLE sys_user_role (
162   id BIGINT NOT NULL AUTO_INCREMENT COMMENT '关联主键',
163   user_id BIGINT NOT NULL COMMENT '用户编号',
164   role_id BIGINT NOT NULL COMMENT '角色编号',
165   create_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
166   update_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP COMMENT '更
      新时间',
167   deleted TINYINT NOT NULL DEFAULT 0 COMMENT '软删除标记',
168   PRIMARY KEY (id),
169   UNIQUE KEY uk_user_role (user_id, role_id),
170   KEY idx_user_role_role (role_id),
171   CONSTRAINT fk_user_role_user FOREIGN KEY (user_id) REFERENCES sys_user(id) ON DELETE CASCADE,
172   CONSTRAINT fk_user_role_role FOREIGN KEY (role_id) REFERENCES sys_role(id) ON DELETE CASCADE
173 ) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci COMMENT='用户角色关联表';
174
175 CREATE TABLE sys_role_permission (
176   id BIGINT NOT NULL AUTO_INCREMENT COMMENT '关联主键',
177   role_id BIGINT NOT NULL COMMENT '角色编号',
178   permission_id BIGINT NOT NULL COMMENT '权限编号',
179   create_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP COMMENT '创建时间',
180   update_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP COMMENT '更
      新时间',
```

五、 系统实现与测试

5.1 本章概述

本章说明系统的开发平台、后端实现、前端实现、数据库实现、自动截图方案和测试结果，并插入代表性运行界面截图。截图由脚本自动生成，能够直接服务于课程设计和答辩展示。

5.2 开发平台和工具选择

表 11: 开发平台和工具选择

层次	技术	选择原因
数据库	MySQL 8	支持事务、外键、索引、utf8mb4 字符集，适合课程设计展示关系模型。
后端	Spring Boot 3 + MyBatis Plus + JWT	便于快速构建 REST API、权限认证、统一返回和数据访问层。
前端	Vue 3 + Vite + Element Plus + ECharts	适合构建课程答辩演示页面、表格表单和统计图表。
截图	Playwright / 系统 Chrome	可自动登录系统并截图，保证报告运行界面一致。
报告	LaTeX + XeLaTeX	中文排版稳定，适合插入长表格、SQL 代码和运行截图。

5.3 后端实现说明

后端采用 Spring Boot 3 分层结构，包名为 `edu.cqupt.visitor`，主要包含 Entity、Mapper、Service、ServiceImpl、Controller、DTO、配置类和安全认证模块。接口统一以 `/api` 为前缀，返回统一结果结构，并通过 JWT 和角色权限控制保护核心接口。

5.4 前端实现说明

前端采用 Vue 3 和 Element Plus，包含登录页、首页统计驾驶舱、访客预约、我的预约、待确认、部门审批、门岗核验、出入校登记、当前在校、超时未离校、黑名单、访客记录、统计报表、用户管理、角色权限、部门管理、校门管理和系统日志等页面。页面字段与后端接口和数据库状态值保持一致。

5.5 数据库实现说明

数据库采用 MySQL 8 作为持久化平台，围绕用户权限、访客预约、审批流转、通行凭证、出入校记录、黑名单、通知消息和操作日志等对象建立关系表。核心表均设置主键、必要外键、业务唯一约束、索引、状态字段和时间字段，以保证数据完整性、查询效率和业务流转可追踪性。

5.6 自动截图说明

系统提供自动化截图能力，用于在统一演示数据基础上记录登录、首页驾驶舱、预约申请、审批处理、门岗核验、出入校登记、黑名单管理、统计报表和系统管理等代表

性界面。该过程保证报告中的运行截图来自同一套业务数据，便于审查系统功能的完整性和一致性。

5.7 测试方法

测试采用数据库初始化测试、接口启动测试、前端页面测试、角色权限测试、核心流程测试、统计报表测试、自动截图测试和 LaTeX 报告编译测试相结合的方式。重点验证数据结构、业务状态、接口响应、页面展示和报告材料之间是否保持一致。

表 12: 系统测试用例摘要

编号	测试名称	操作/输入	预期结果
TC-01	数据库初始化	执行 scripts/init_database.sh	成功创建 cqupt_visitor_system 并导入演示数据
TC-02	后端启动	执行 scripts/run_backend.sh	Swagger 可访问，核心接口返回统一结果
TC-03	前端启动	执行 scripts/run_frontend.sh	登录页可访问，页面资源加载正常
TC-04	登录认证	使用 admin/123456 登录	返回 Token 并进入首页
TC-05	角色菜单	切换 teacher01、approver01、guard01、manager01	菜单与角色权限匹配
TC-06	预约申请	访客提交预约	生成 visit_apply 记录并进入待确认状态
TC-07	黑名单检查	使用黑名单手机号或证件号预约/核验	系统拦截或提示风险
TC-08	被访人确认	teacher01 处理待确认预约	写入 approval_record 并更新状态
TC-09	部门审批	approver01 审批本部门预约	审批通过生成通行凭证，拒绝则终止流程
TC-10	门岗核验	guard01 输入通行码或预约编号	返回允许/拒绝入校结果
TC-11	入校登记	核验通过后登记入校	生成或更新 access_record，状态为已入校
TC-12	离校登记	对已入校访客办理离校	记录离校时间，禁止重复离校
TC-13	统计报表	访问首页和统计报表页	趋势、排行、状态分布等图表均有数据

TC-14	运行界面采集	执行自动化演示流程	形成 19 张代表性运行界面截图
TC-15	LaTeX 报告	执行 xelatex main.tex 两次	生成 report-latex/main.pdf

5.8 代表性运行界面

以下运行界面选取系统中最具代表性的功能页面，展示登录认证、首页统计、访客预约、审批处理、门岗核验、出入校登记、风险管理、统计分析和系统维护等功能的实现效果。

展示系统标题、登录表单、测试账号提示和校园蓝视觉风格。



图 13: 登录页

展示今日访客数、当前在校访客、超时未离校、部门访客排行、校门通行统计等核心指标。

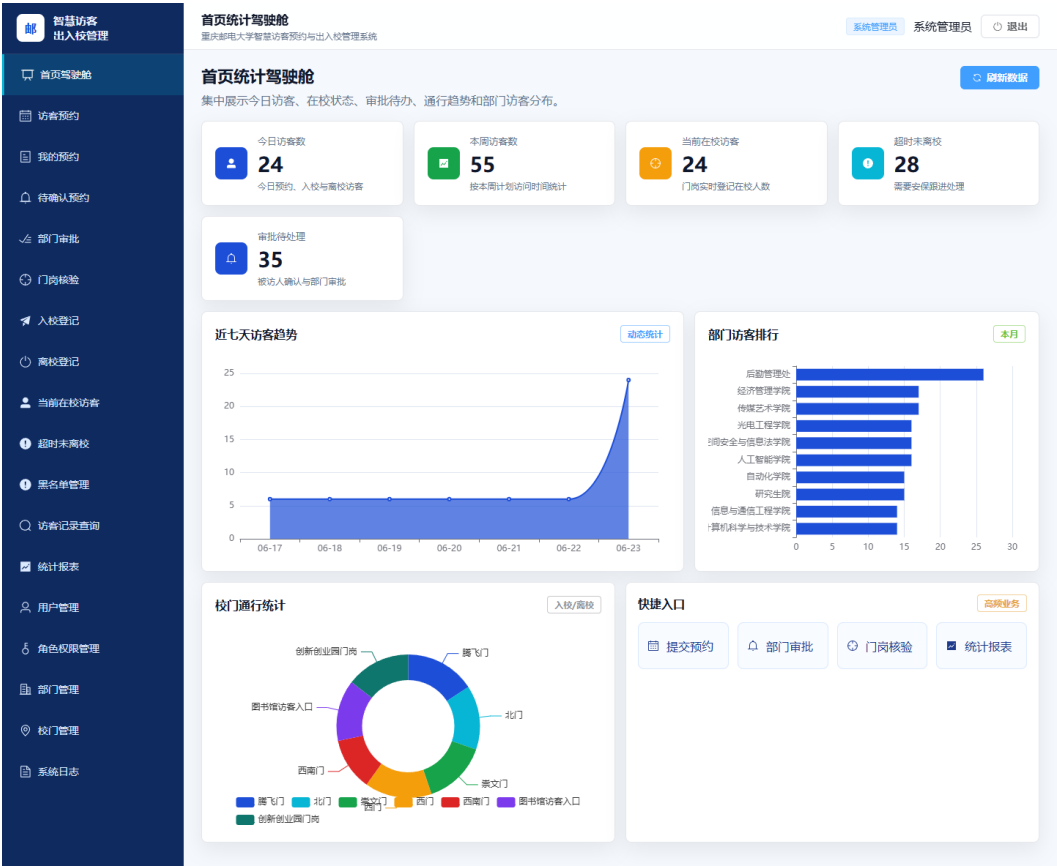


图 14：首页统计驾驶舱

展示访客填写来访事由、被访人、访问时间、车辆和随行人员等预约申请信息。



图 15：访客预约申请页

展示预约记录、审批状态和访问状态，便于说明访客端状态查询功能。

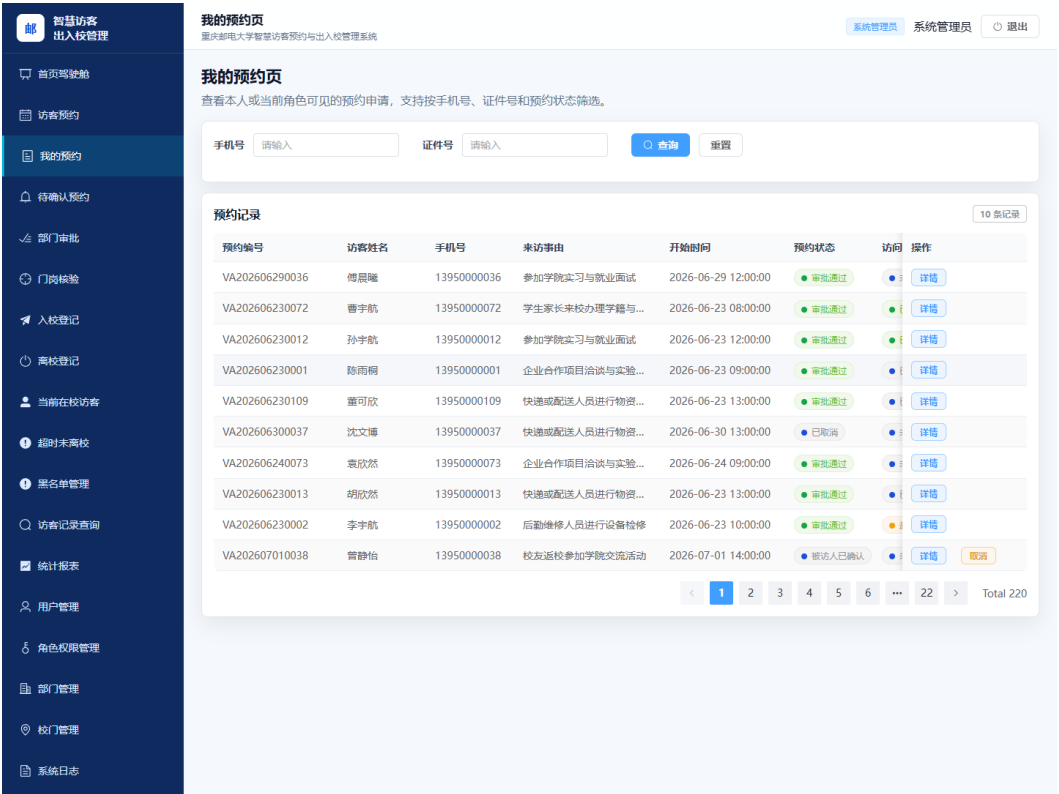


图 16: 我的预约页

展示被访人确认或拒绝预约申请的业务界面。

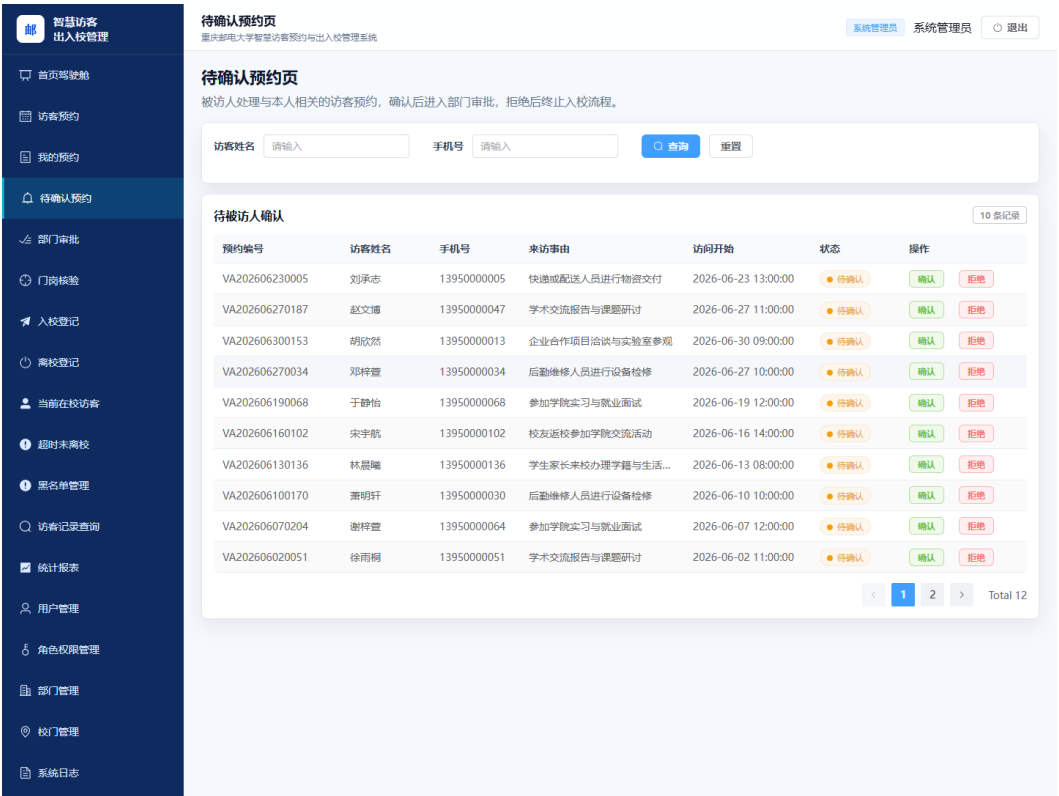


图 17: 被访人待确认页

展示部门审批人员处理预约申请、填写审批意见和查看审批状态的功能。

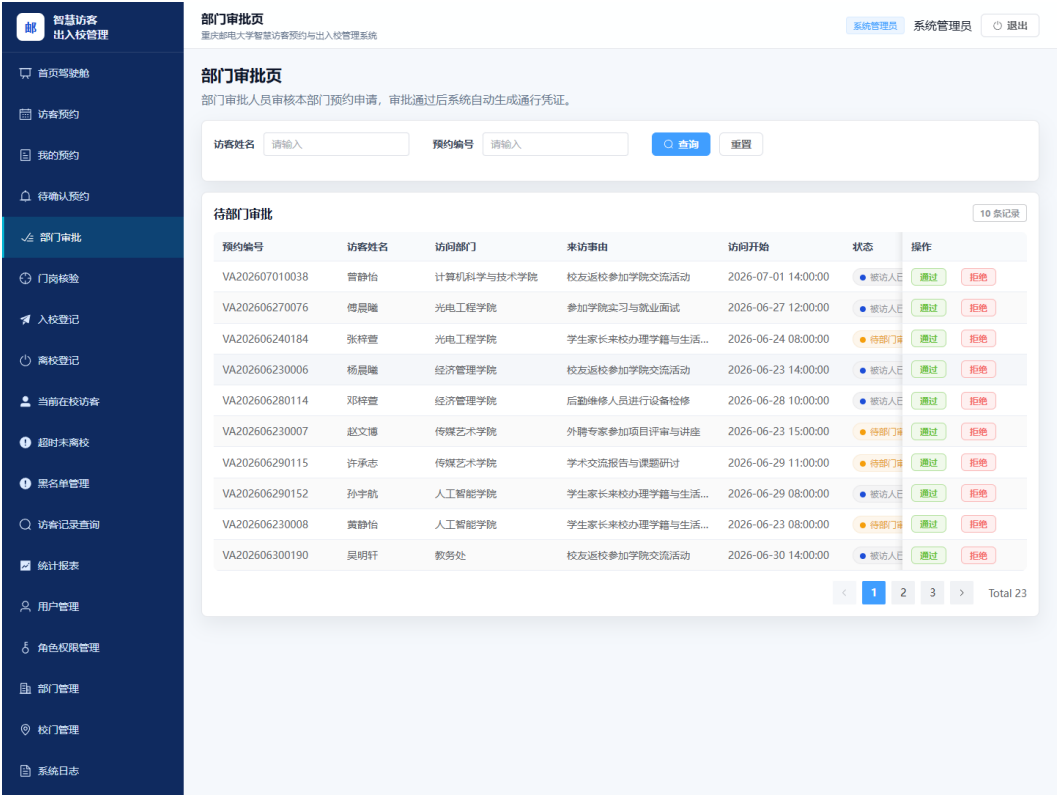


图 18：部门审批页

展示门岗按预约编号、手机号、证件号或通行码进行访客核验的界面。

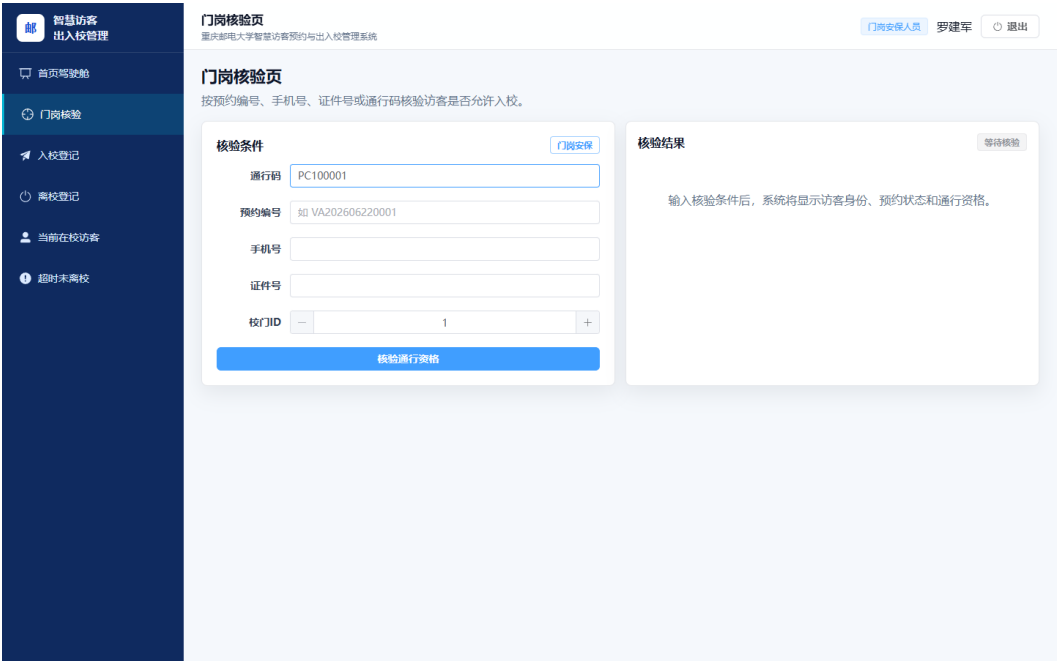


图 19：门岗核验页

展示核验通过后记录入校时间、校门和安保人员的登记页面。



图 20: 入校登记页

展示已入校访客离校登记和访问状态更新功能。



图 21: 离校登记页

展示当前已入校但尚未离校的访客列表。

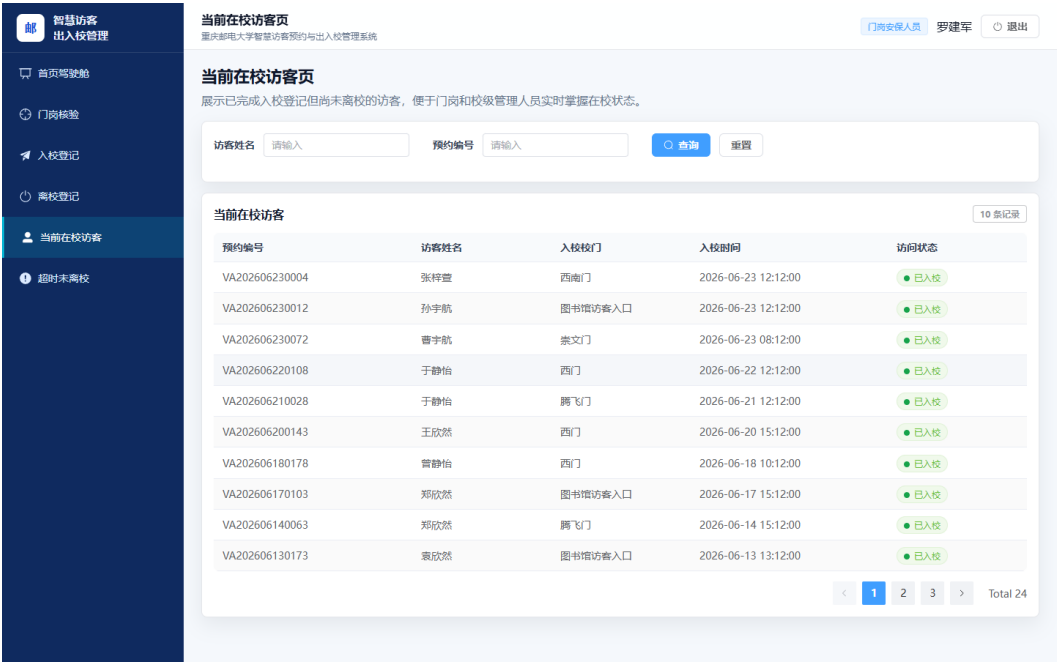


图 22: 当前在校访客页

展示超过预计离校时间仍未离校的访客及风险提示。

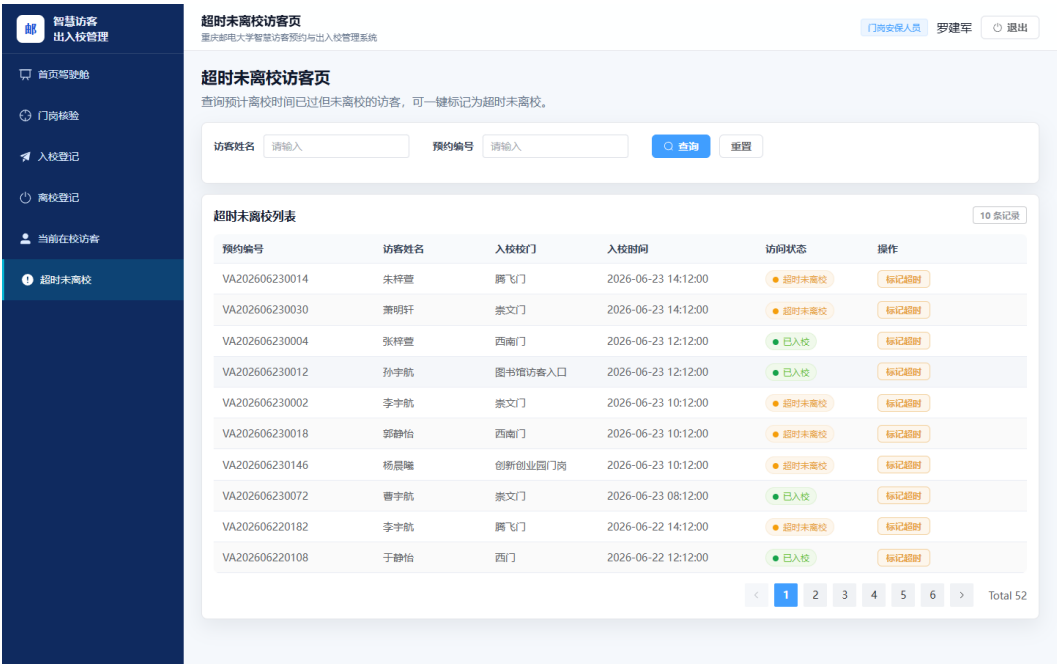


图 23: 超时未离校访客页

展示身份异常、多次超时、虚假预约等风险访客黑名单管理。

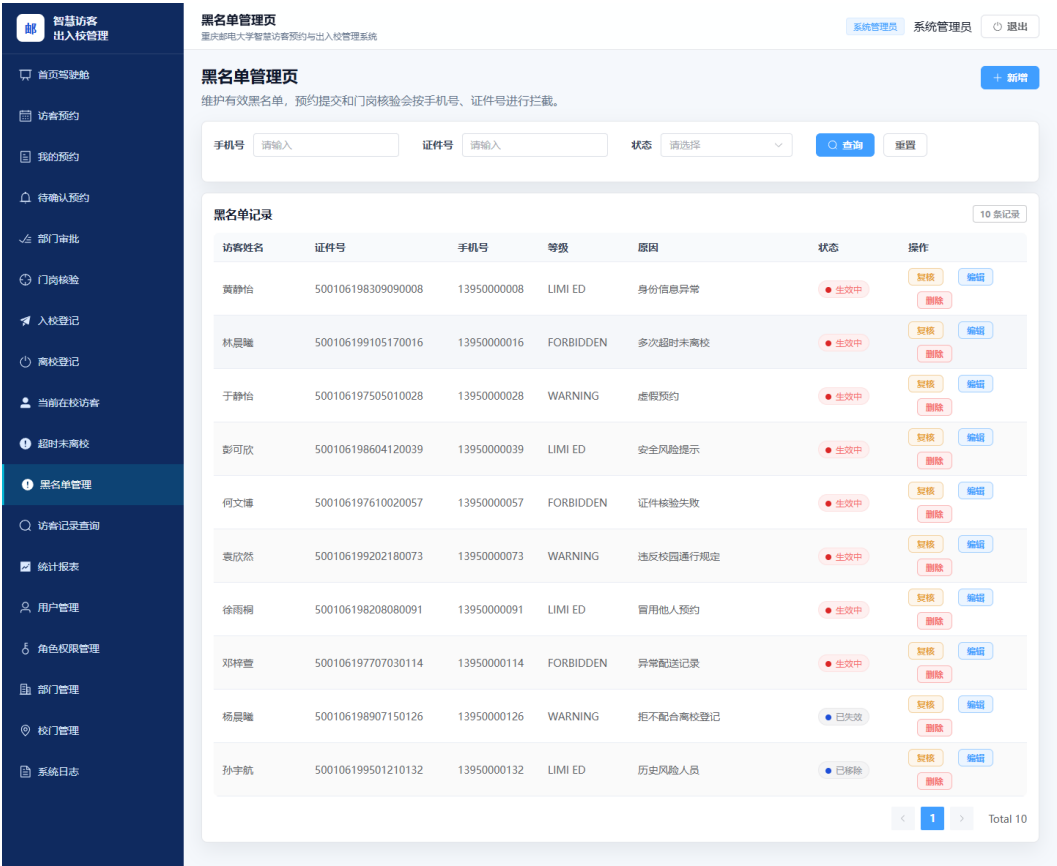


图 24：黑名单管理页

展示按访客、部门、时间和状态查询全校访客记录的功能。

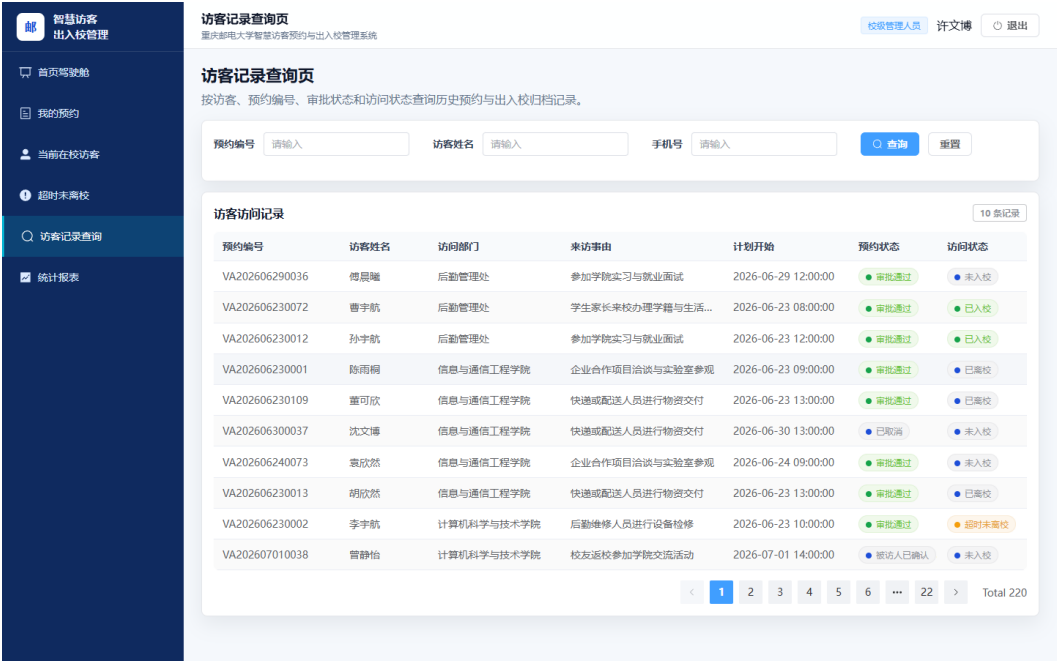


图 25：访客记录查询页

展示近七天访客趋势、部门访客排行、校门通行统计、审批状态和黑名单风险等图

表。

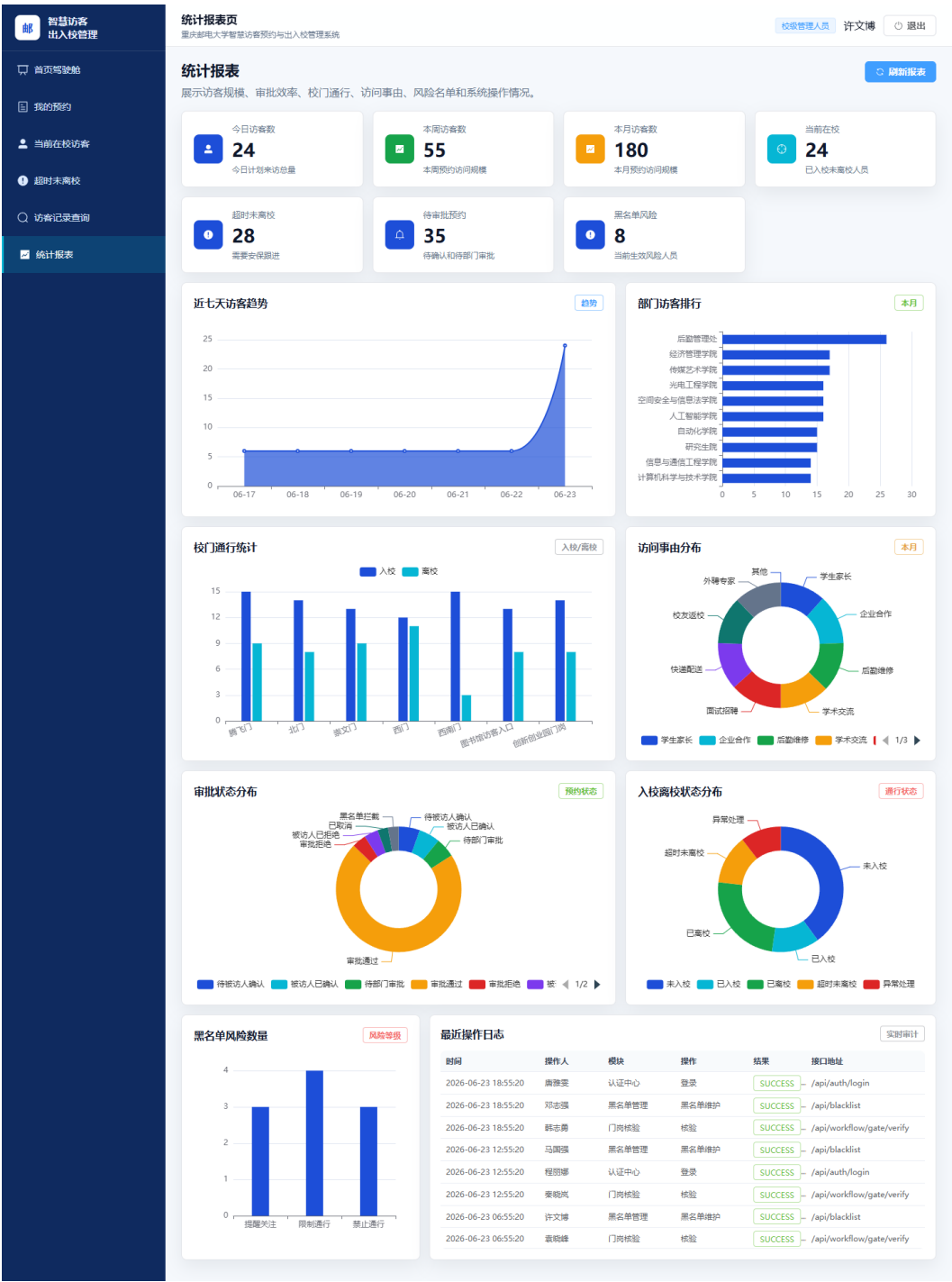


图 26: 统计报表页

展示系统用户账号、角色类型、所属部门和账号状态管理。

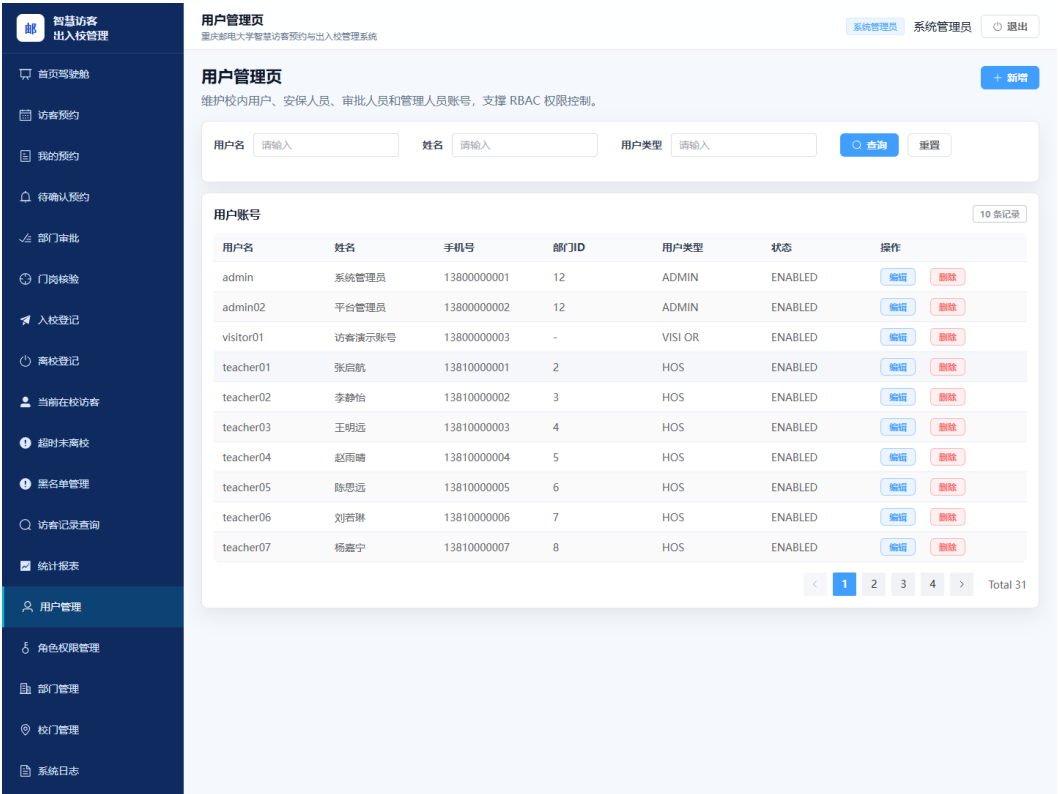


图 27: 用户管理页

展示系统角色和权限配置，支撑 RBAC 权限控制。

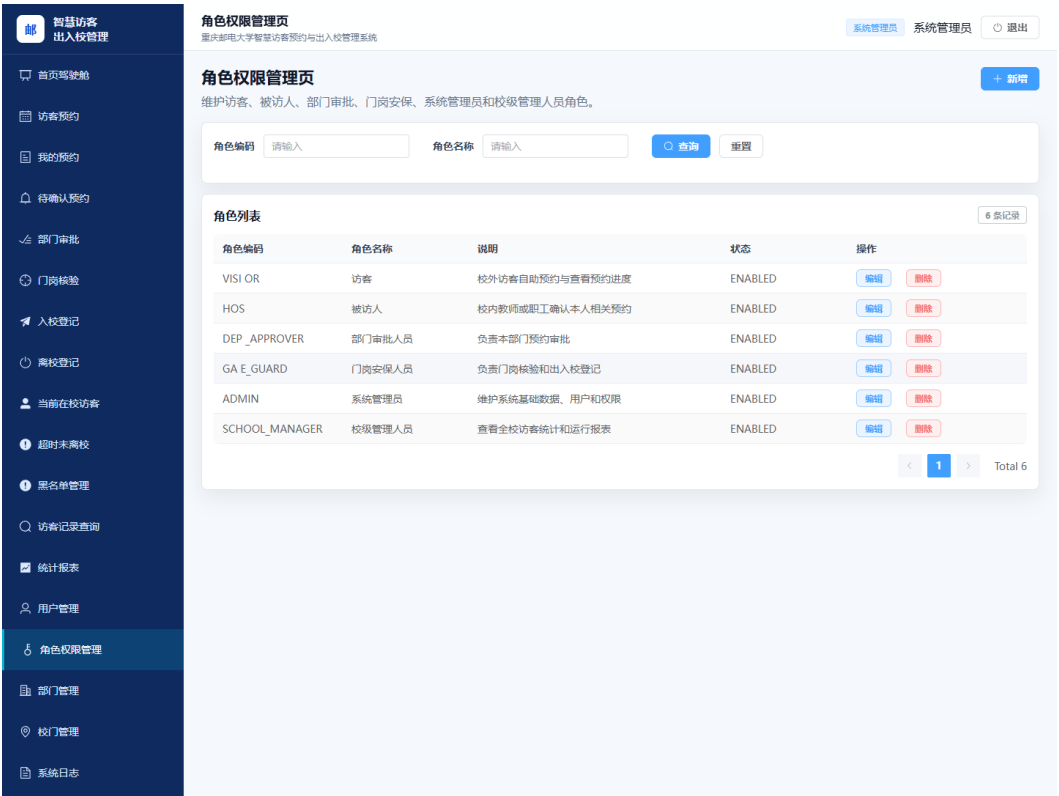


图 28: 角色权限管理页

展示学院、职能部门等组织结构基础数据维护。

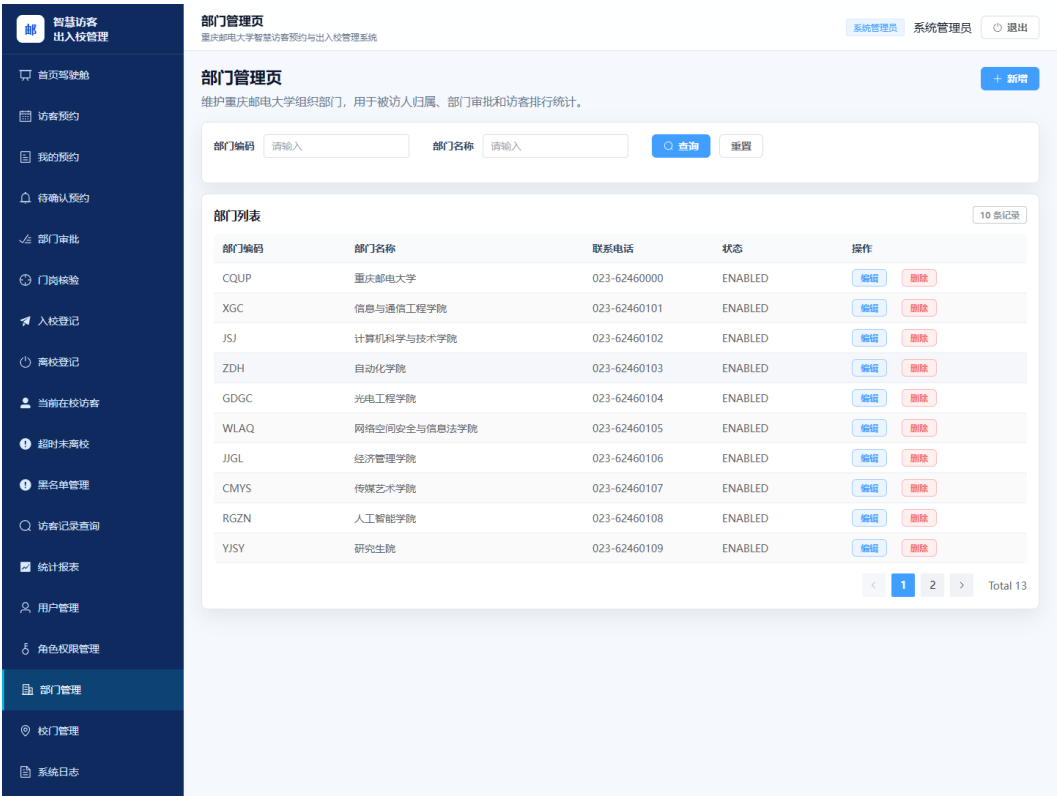


图 29：部门管理页

展示腾飞门、北门、崇文门等校门基础数据维护。

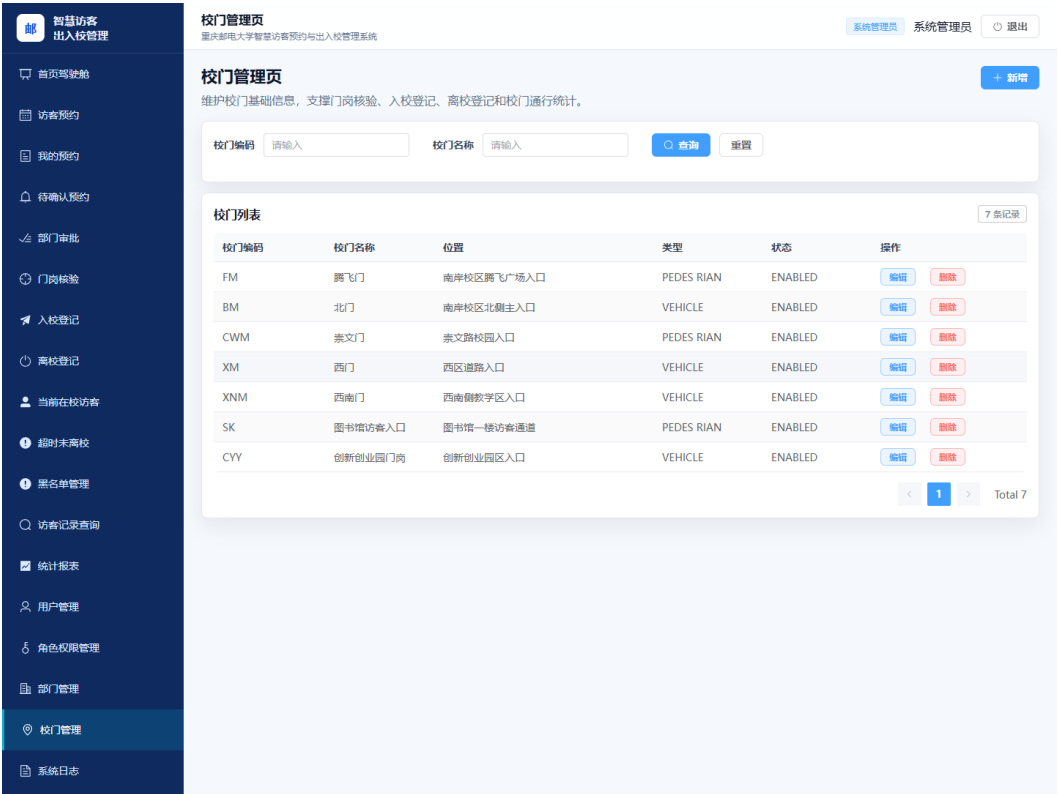


图 30: 校门管理页

展示登录、预约、审批、出入校登记、黑名单维护和报表查询等关键操作日志。

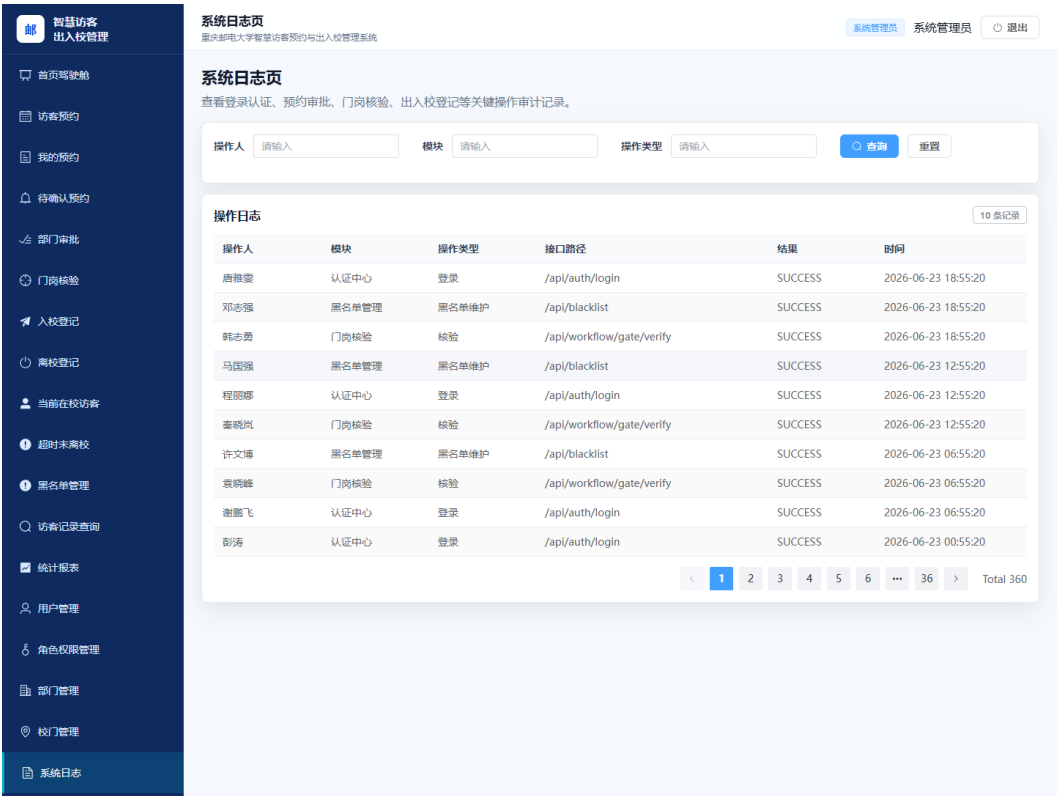


图 31: 系统日志页

六、 总结

本课程设计围绕“重庆邮电大学智慧访客预约与出入校管理系统”完成了从需求分析、概念结构设计、逻辑结构设计、MySQL 建库建表、后端接口、前端页面、权限控制、核心业务流程、统计报表、自动截图到 LaTeX 报告生成的完整实践。系统以访客预约为主线，将访客信息、预约申请、审批记录、通行凭证、出入校记录、黑名单、用户权限和系统日志统一纳入数据库管理，能够较好地体现数据库系统在校园安全管理场景中的建模、约束、查询和应用价值。

在数据库设计方面，系统明确了实体、关系、主键、外键、唯一约束、索引和状态字段，核心表结构能够覆盖从预约提交到访问归档的完整流程。在系统实现方面，后端采用 Spring Boot 3、MyBatis Plus、MySQL 和 JWT，前端采用 Vue 3、Element Plus、Axios、Vue Router 和 ECharts，页面能够展示预约、审批、核验、出入校、黑名单、统计报表和系统管理等核心功能。在自动化方面，项目提供 Playwright 截图脚本和 LaTeX 报告项目，便于将运行界面、SQL、图表和测试说明整合为最终课程设计报告。

总体来看，本系统从数据库课程设计要求出发，完成了系统定义、需求分析、数据流程分析、概念结构设计、逻辑结构设计、SQL 详细设计、系统实现和测试验证等环节。设计结果能够覆盖校园访客预约与出入校管理的主要业务场景，报告中的图表、关系模式、SQL 查询和运行界面能够共同说明系统的数据组织方式与实际运行效果。